

# Travail de Fin d'Études

Module de gestion des bénévoles et de réservation des événements

**Étudiant :**

Alain Youndjeu Tchouapi

**Encadrement :**

Marie-Christine Namur  
(professeure)

Bachelier en Informatique de Gestion

Application *Terra Sana*

« Recueillir un besoin réel, le traduire en application fonctionnelle. »

**Année académique : 2025 – 2026**

*Travail de fin d'études en vue de l'obtention du titre de bachelier en informatique de gestion*

## Remerciements

---

Avant d'entrer dans le corps de ce rapport, je tiens à remercier toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce travail de fin d'études.

**Mes premiers remerciements vont à Madame Marie-Christine Namur**, ma promotrice, pour son encadrement, sa disponibilité et la rigueur de ses retours tout au long de la phase d'analyse et de développement de ce module. Ses conseils ont directement orienté plusieurs choix de conception présentés dans ce rapport.

**Je remercie également Monsieur Didier Seraye**, Responsable Administratif de Terra Sana ASBL et mon maître de stage, pour la confiance qu'il m'a accordée durant le stage puis pour sa disponibilité lors des échanges qui ont permis de cadrer précisément les besoins réels de l'association en matière de gestion des bénévoles et des événements.

Ma reconnaissance va aussi à l'ensemble du corps professoral de l'EAFC Uccle, pour la qualité de la formation dispensée durant ces trois années de Bachelier en Informatique de Gestion, et pour les bases méthodologiques — analyse, modélisation, bonnes pratiques de développement — sans lesquelles ce travail n'aurait pas la même rigueur.

Enfin, je remercie mes proches et mes camarades de promotion pour leur soutien constant durant cette dernière année d'études, en particulier durant les semaines les plus intenses de rédaction et de développement.

## Résumé

---

Terra Sana ASBL, association bruxelloise active depuis 2019 dans l'alimentation locale et le circuit court, gérait jusqu'ici ses bénévoles et ses événements de façon entièrement manuelle — fichiers Excel partagés, confirmations envoyées une à une par email, aucune trace exploitable de la participation des bénévoles. Ce travail de fin d'études développe, en continuité du site vitrine réalisé durant le stage effectué au sein de l'association, un module complet de gestion des bénévoles et des événements : création de compte et espace personnel, inscription aux événements avec liste d'attente automatique, validation par l'administrateur, notifications par email, retours post-événement, niveaux de fidélité calculés automatiquement et attestations de participation téléchargeables.

L'analyse s'appuie sur vingt-cinq règles de gestion établies à partir d'entretiens directs avec le responsable administratif de l'association, un diagramme de classes à huit entités et une base de données MySQL normalisée. L'implémentation repose sur une architecture 3-tiers Spring Boot / React / MySQL, avec une authentification JWT à deux rôles distincts (bénévole et administrateur), un tableau de bord administrateur entièrement traduit en français, anglais et néerlandais, et une suite de mécanismes automatisés (attribution de photos par mots-clés, envoi d'e-mails asynchrone, génération de PDF) documentés dans le dossier technique. Le module est développé et démontré en environnement local, conformément au périmètre validé avec l'encadrement académique.

**Mots-clés :** Spring Boot, React, JWT, MySQL, gestion associative, bénévolat, application web multilingue.

# Table des matières

---

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>Remerciements.....</b>                                    | <b>2</b>  |
| <b>Résumé.....</b>                                           | <b>3</b>  |
| <b>I. Introduction.....</b>                                  | <b>5</b>  |
| 1.1. Présentation de Terra Sana ASBL.....                    | 5         |
| 1.2. Contexte du stage et travail réalisé avant le TFE.....  | 5         |
| 1.3. Problématique.....                                      | 6         |
| 1.4. Objectifs et périmètre du TFE.....                      | 6         |
| 1.5. Justification du périmètre retenu.....                  | 6         |
| 1.6. Sources d'information.....                              | 7         |
| <b>II. Dossier d'analyse.....</b>                            | <b>8</b>  |
| 2.1. Cahier des charges.....                                 | 8         |
| 2.2. Persistance des données.....                            | 15        |
| 2.3. Autres diagrammes UML.....                              | 20        |
| <b>III. Dossier technique.....</b>                           | <b>24</b> |
| 3.1. Choix technologiques.....                               | 24        |
| 3.2. Architecture globale.....                               | 25        |
| 3.3. Fonctionnalités transversales notables.....             | 27        |
| 3.4. Présentation des interfaces.....                        | 30        |
| 3.5. Sécurité.....                                           | 41        |
| 3.6. Environnement de développement et de démonstration..... | 43        |
| <b>IV. Conclusion.....</b>                                   | <b>45</b> |
| 4.1. Apports personnels.....                                 | 45        |
| 4.2. Difficultés rencontrées.....                            | 46        |
| 4.3. Regard critique sur le travail réalisé.....             | 46        |
| 4.4. Perspectives.....                                       | 47        |
| 4.5. Mot de fin.....                                         | 47        |
| <b>V. Glossaire.....</b>                                     | <b>48</b> |
| <b>VI. Bibliographie / Webographie.....</b>                  | <b>49</b> |
| <b>VII. Annexes.....</b>                                     | <b>51</b> |

# I. Introduction

## 1.1. Présentation de Terra Sana ASBL

Terra Sana ASBL est une association belge sans but lucratif, active à Bruxelles depuis 2019 dans les domaines de l'alimentation locale et saine, du circuit court et du bien-être communautaire. Son siège se situe au 53/3, 1200 Woluwe-Saint-Lambert. L'association organise régulièrement des ateliers, des marchés bio, des collectes et des activités communautaires, animés en grande partie par des bénévoles engagés.

Pour gérer ses différents pôles d'activité, l'association dispose de douze applications internes spécialisées (comptabilité, gestion de factures, agenda personnel, bibliothèque, etc.), dont l'accès est centralisé depuis un site web développé durant mon stage.

La mission de Terra Sana repose sur quatre piliers, mis en avant sur la page d'accueil du site (section 3.4.1) : une agriculture naturelle privilégiant les circuits courts et limitant l'usage d'intrants chimiques, une communauté inclusive ouverte à tous sans condition, un soutien direct aux producteurs locaux, et un impact reconnu localement, notamment par la Région bruxelloise qui figure parmi ses partenaires (section 3.4.7). L'activité de l'association repose structurellement sur l'engagement bénévole : sans une base de bénévoles mobilisables rapidement pour chaque événement, la plupart des activités de terrain de Terra Sana ne pourraient simplement pas avoir lieu — ce qui explique pourquoi la gestion de ces bénévoles, plutôt qu'un autre module, s'est imposée comme le sujet le plus porteur pour ce TFE (section 1.5).

## 1.2. Contexte du stage et travail réalisé avant le TFE

Mon stage s'est déroulé au sein de Terra Sana ASBL du 25 mars au 20 mai 2026, sous la supervision de Monsieur Didier Seraye, Responsable Administratif de l'association. Avant ce stage, Terra Sana ne possédait aucune présence numérique : la communication avec les bénévoles et le public se faisait exclusivement par téléphone et par email, et la gestion administrative reposait entièrement sur des fichiers Excel et des documents papier.

Durant cette période, j'ai conçu et développé de zéro un site web complet servant de vitrine institutionnelle et de hub centralisé pour les douze applications internes de l'association. Le tableau ci-dessous résume les composants livrés à la fin du stage, qui constituent la fondation technique sur laquelle s'appuie le présent TFE.

| Composant               | Description                                                                                                                                                                                  |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Site public multilingue | 12 pages React (Accueil, À propos, Projets, Détail projet, Blog, Contact, Bénévolat, Sponsors, Confidentialité, Conditions, Cookies, Aide/FAQ), trilingue FR/EN/NL avec sélecteur de langue. |
| Hub des 12 applications | Affichage dynamique de l'ensemble des applications internes de l'association, avec recherche et filtres par catégorie.                                                                       |
| Espace administrateur   | Connexion sécurisée (JWT 24h) et dashboard permettant la gestion des projets, des articles de blog, des messages de contact et le changement de mot de passe.                                |
| API REST backend        | 17 endpoints exposés sur 4 ressources (auth, projects, posts, contact), sécurisés par JWT et rôle administrateur.                                                                            |
| Base de données         | Base MySQL de 4 tables : admin, project, blog_post et contact_message.                                                                                                                       |
| Envoi d'e-mails         | Réponse aux messages de contact via Gmail SMTP.                                                                                                                                              |

Tableau 1 — Composants livrés à la fin du stage (avant le TFE)

### 1.3. Problématique

À l'issue du stage, Terra Sana continuait de gérer ses bénévoles et ses activités de façon entièrement manuelle. Cette organisation engendrait plusieurs difficultés concrètes, identifiées avec Monsieur Seraye :

- des doublons et des pertes d'information lorsque plusieurs personnes modifiaient les mêmes fichiers Excel ;
- aucune vue d'ensemble des places disponibles sur les activités, d'où des événements surchargés ou au contraire sous-remplis ;
- des confirmations et des relances envoyées une à une par email, un travail répétitif et chronophage pour l'équipe ;
- aucun historique de participation, ni moyen de remercier les bénévoles les plus actifs ou de leur délivrer une attestation.

Ces difficultés ne sont pas propres à Terra Sana : elles sont typiques de toute petite structure associative dont l'activité croît plus vite que ses outils de gestion. Ce qui fonctionnait raisonnablement avec une poignée de bénévoles et deux événements par mois devenait, au moment du cadrage de ce TFE, un frein concret à mesure que le nombre d'activités et de bénévoles inscrits augmentait — chaque nouveau bénévole ou événement alourdissant un peu plus une charge de coordination déjà tenue à la limite du gérable.

Le module développé dans le cadre de ce TFE vise précisément à centraliser et automatiser ces tâches : un espace bénévole en ligne, une gestion des événements avec places et liste d'attente, des emails de confirmation automatiques et un historique exploitable. **Ce module est développé après le stage, entièrement par mes soins et sans code préexistant : à la fin du stage, aucune ligne de code n'existait pour ces fonctionnalités.**

### 1.4. Objectifs et périmètre du TFE

Ce travail poursuit un double objectif : répondre à un besoin réel et documenté de l'association, tout en démontrant la maîtrise d'un cycle complet d'analyse et de développement — du cahier des charges jusqu'à une application fonctionnelle, en environnement local. Le module à livrer couvre, dans les grandes lignes, deux volets complémentaires : côté bénévole, un espace personnel permettant de s'inscrire aux événements de l'association et de suivre sa propre participation ; côté administrateur, un outil de pilotage permettant de créer ces événements et d'en gérer les inscriptions. La liste complète et détaillée des fonctionnalités, par catégorie d'utilisateur, fait l'objet du cahier des charges (section 2.1.2).

Le développement et la démonstration de l'application sont réalisés en environnement local (WAMP Server sous Windows) : aucun déploiement en ligne n'est prévu dans le cadre de ce TFE, ce point est détaillé en section III.6.

### 1.5. Justification du périmètre retenu

Parmi les douze applications internes que gère Terra Sana, plusieurs domaines auraient pu constituer le sujet de ce TFE : la comptabilité, la gestion de stock, ou encore la planification interne des tâches administratives. Le choix s'est porté sur la gestion des bénévoles et des événements pour trois raisons concrètes, établies directement avec Monsieur Seraye durant les entretiens de cadrage.

- C'est le point de friction le plus régulièrement cité par l'association : contrairement à la comptabilité, tenue par une seule personne, la gestion des événements implique plusieurs bénévoles et l'administrateur simultanément, ce qui multiplie les erreurs de coordination par fichier Excel partagé.
- Le périmètre est suffisamment autonome pour être livré et démontré indépendamment des autres applications internes de l'association, sans dépendance à un système existant à intégrer.
- Il couvre naturellement l'ensemble des briques attendues d'un TFE en Bachelier Informatique de Gestion : authentification à deux rôles, modélisation de données relationnelles avec contraintes réelles, logique métier non triviale (liste d'attente, niveaux de fidélité) et génération de documents PDF.

Certaines fonctionnalités, envisagées un temps durant l'analyse, ont été volontairement écartées du périmètre livré pour rester réalisable dans les délais de la 2ème session : la synchronisation avec un calendrier externe (Google Calendar), la mise en place de notifications push, et une gestion multi-administrateurs avec des droits différenciés. Ces pistes sont reprises comme perspectives d'évolution en section IV.4.

## 1.6. Sources d'information

Les informations utilisées pour concevoir ce module proviennent de plusieurs sources complémentaires, réunies pendant et après le stage :

- des entretiens avec Monsieur Didier Seraye afin de comprendre les besoins réels, le fonctionnement quotidien de l'association et les points de friction dans la gestion actuelle des bénévoles et des événements ;
- l'analyse des fichiers Excel utilisés à ce jour, pour identifier les données à structurer (identité des bénévoles, historiques d'ateliers, listes d'inscrits) ;
- la documentation officielle des technologies retenues (Spring Boot, React, Spring Security, JavaMail, iText) ainsi que les bonnes pratiques REST et de sécurité en vigueur ;
- les consignes de l'EAFC Uccle pour la rédaction du présent rapport et la structuration du dossier d'analyse.

Ces sources ne sont pas mobilisées de façon isolée : les entretiens avec Monsieur Seraye ont fixé les grandes lignes du besoin (section 1.3), les fichiers Excel existants ont permis de vérifier concrètement quels champs de données étaient réellement suivis par l'association avant de les faire figurer dans le dictionnaire de données (section 2.2.2), et la documentation officielle des technologies a servi de référence à chaque choix d'implémentation détaillé dans le dossier technique (section III), plutôt que d'être consultée après coup pour justifier un choix déjà fait.

## II. Dossier d'analyse

Le dossier d'analyse constitue le cœur de la réflexion menée avant le développement. Il formalise les besoins du module, définit précisément les cas d'utilisation, modélise les données et établit la structure physique de la base de données. Cette section couvre successivement le cahier des charges, le diagramme de cas d'utilisation, les règles de gestion, puis la persistance des données (diagramme de classes, dictionnaire de données, schéma SQL).

### 2.1. Cahier des charges

#### 2.1.1. Objectifs et contexte d'utilisation

Le module à livrer étend le site vitrine développé pendant le stage en y intégrant une gestion complète des bénévoles et des événements de Terra Sana ASBL. Il s'adresse à deux catégories d'utilisateurs bien distinctes, aux besoins différents : les bénévoles, qui consultent et s'inscrivent aux événements depuis chez eux, et l'administrateur unique de l'association, qui pilote l'ensemble depuis un tableau de bord.

Contrairement à une application e-commerce classique, il n'y a ici ni panier, ni paiement, ni gestion de stock : la ressource centrale à gérer est la place disponible à un événement, avec une contrainte de capacité (maxPlaces) et un mécanisme de liste d'attente lorsque cette capacité est atteinte. C'est cette logique de réservation à capacité limitée, combinée à un système de validation manuelle par l'administrateur, qui structure l'ensemble des règles de gestion détaillées en section 2.1.5.

#### 2.1.2. Catégories d'utilisateurs et fonctionnalités

Trois profils interagissent avec le système : le Visiteur, non authentifié, qui peut consulter le site et les événements publics ; le Bénévole, un visiteur ayant créé un compte, qui accède à son espace personnel et peut s'inscrire aux événements ; et l'Administrateur, rôle unique côté Terra Sana, qui gère l'ensemble des événements, des bénévoles et des inscriptions. Le tableau ci-dessous détaille les fonctionnalités livrées pour chacun.

| Module                   | Fonctionnalité                                                                         | Rôle     |
|--------------------------|----------------------------------------------------------------------------------------|----------|
| Compte bénévole          | Création de compte, connexion (JWT dédié), mot de passe oublié, modification du profil | Bénévole |
| Événements               | Consultation des événements à venir et passés, avec places restantes                   | Visiteur |
| Inscriptions             | S'inscrire, rejoindre la liste d'attente si complet, se désinscrire librement          | Bénévole |
| Historique               | Consultation de l'historique de participation, téléchargement d'attestation PDF        | Bénévole |
| Avis                     | Laisser un avis noté (1 à 5) après un événement terminé auquel on a participé          | Bénévole |
| Niveau de fidélité       | Calcul automatique Bronze / Argent / Or selon les participations confirmées            | Système  |
| Gestion des événements   | Créer, modifier, changer le statut, supprimer un événement                             | Admin    |
| Gestion des inscriptions | Valider ou refuser une inscription, voir la liste d'attente par événement              | Admin    |
| Communication            | Email de confirmation automatique, envoi d'emails groupés aux inscrits                 | Admin    |
| Export                   | Export PDF de la liste des inscrits à un événement                                     | Admin    |
| Gestion des bénévoles    | Liste complète avec filtres par compétence, disponibilité et langue                    | Admin    |
| Tableau de bord          | Statistiques (bénévoles actifs, événements, taux de participation),                    | Admin    |

| Module | Fonctionnalité | Rôle |
|--------|----------------|------|
|        | graphiques     |      |

Tableau 2 — Fonctionnalités du module, par catégorie d'utilisateur

Le compte bénévole (RG-01 à RG-04) constitue le point d'entrée : au-delà de l'email et du mot de passe, il collecte des informations facultatives — compétences, disponibilités, ville — que l'administrateur utilise ensuite pour cibler ses recherches lorsqu'il prépare un événement nécessitant un profil particulier (par exemple des compétences culinaires pour un atelier cuisine).

La consultation des événements est volontairement ouverte à tout visiteur, sans compte : Terra Sana souhaitait que la vitrine de ses activités reste visible publiquement, la création de compte n'intervenant qu'au moment de vouloir réellement s'inscrire. Cette distinction se reflète directement dans SecurityConfig (section 3.5.2), où les routes de lecture des événements sont publiques alors que l'inscription elle-même exige un jeton.

Les inscriptions et la liste d'attente (RG-09 à RG-14) forment le cœur fonctionnel du module, détaillé pas à pas dans les diagrammes de séquence de la section 2.3. Le bénévole y garde la main sur sa désinscription à tout moment avant le début de l'événement, tandis que la confirmation d'une inscription reste toujours un acte explicite de l'administrateur — le système ne confirme jamais automatiquement une première inscription, seule une promotion depuis la liste d'attente l'est (RG-13).

La gestion des avis et le niveau de fidélité (RG-15 à RG-20) ajoutent une dimension incitative absente du fonctionnement précédent, entièrement manuel : un bénévole voit concrètement sa progression, et Terra Sana dispose d'un indicateur simple pour identifier ses bénévoles les plus engagés sans avoir à dépouiller des fichiers Excel.

Côté administration, la gestion des événements, des inscriptions et des bénévoles (dernières lignes du tableau) reproduit numériquement ce que Monsieur Seraye effectuait jusque-là à la main : créer un événement, suivre qui s'y inscrit, relancer par email, exporter une liste pour l'impression. Le tableau de bord ajoute une couche que le processus manuel ne pouvait pas offrir : une vue statistique agrégée de l'activité de l'association sur les derniers mois.

### 2.1.3. Besoins non-fonctionnels

Au-delà des fonctionnalités elles-mêmes, plusieurs exigences transversales conditionnent la qualité du module. Elles sont résumées dans le tableau ci-dessous et se retrouvent, pour la plupart, traduites directement en règles de gestion en section 2.1.5.

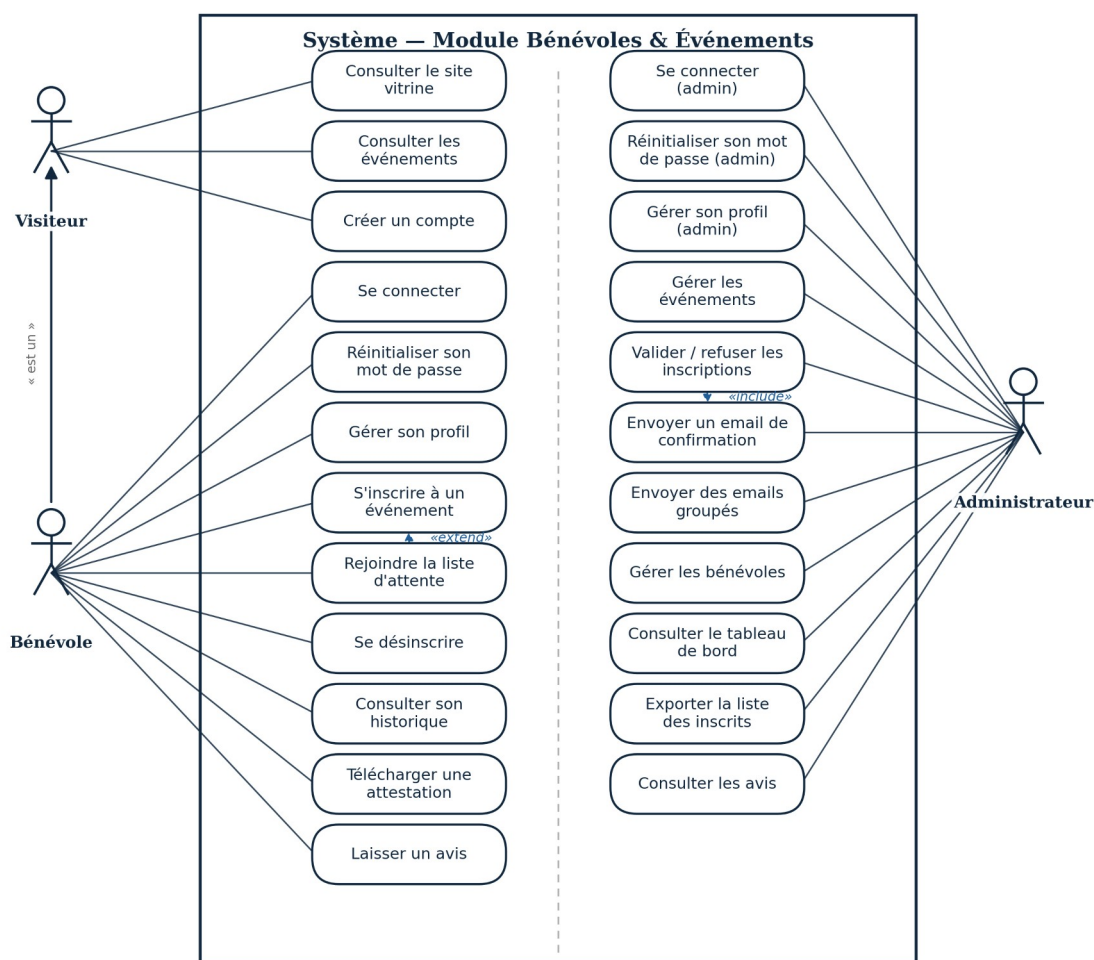
| Catégorie      | Exigence                  | Détail                                                                                                                                                        |
|----------------|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sécurité       | Authentification JWT      | Chaque requête vers une ressource protégée porte un jeton signé ; les routes publiques sont limitées à l'inscription, la connexion et la consultation du site |
| Sécurité       | Isolation des données     | Un bénévole ne peut accéder ou agir que sur ses propres inscriptions, avis et informations de profil                                                          |
| Sécurité       | Hachage des mots de passe | Aucun mot de passe n'est jamais stocké ni transmis en clair (BCrypt)                                                                                          |
| Fiabilité      | Cohérence des places      | Le nombre de places disponibles affiché correspond toujours au nombre réel d'inscriptions confirmées, recalculé à la demande                                  |
| Maintenabilité | Architecture en couches   | Séparation stricte Controller / Service / Repository, propice à l'évolution indépendante de chaque module fonctionnel                                         |

| Catégorie     | Exigence              | Détail                                                                                                         |
|---------------|-----------------------|----------------------------------------------------------------------------------------------------------------|
| Ergonomie     | Interface multilingue | Le site, y compris le dashboard administrateur, est utilisable en français, anglais et néerlandais             |
| Disponibilité | Environnement local   | L'application doit rester pleinement fonctionnelle en démonstration locale, sans dépendance à un service cloud |

Tableau 3 — Besoins non-fonctionnels du module

#### 2.1.4. Diagramme de cas d'utilisation

Le diagramme ci-dessous représente les interactions entre les trois acteurs identifiés et les fonctionnalités offertes par le module. Le Bénévole étend le Visiteur par généralisation (« est un »), puisque tout bénévole reste un visiteur capable de consulter le site public ; le cas d'utilisation « Valider / refuser les inscriptions » inclut systématiquement l'envoi d'un email de confirmation.



Côté bénévole, le diagramme distingue volontairement deux blocs de cas d'utilisation reliés au même acteur : les cas accessibles dès la consultation du site (consulter le site vitrine, consulter les événements, créer un compte) et ceux qui exigent d'être connecté (gérer son profil, s'inscrire, se désinscrire, laisser un avis, télécharger une attestation). Cette distinction reflète directement la séparation des routes publiques et protégées définie dans SecurityConfig (section 3.5.2) : rien dans le diagramme ne suppose une contrainte technique qui ne soit pas également appliquée dans le code.

Côté administration, le cas « Valider / refuser les inscriptions » inclut (relation «include») l'envoi d'un email de confirmation : ce sous-cas n'est jamais déclenché seul, il fait partie intégrante de la validation elle-même, conformément à RG-12. Le cas « S'inscrire à un événement » est quant à lui étendu (relation «extend») par « Rejoindre la liste d'attente », qui ne s'active que dans la situation particulière où l'événement est déjà complet — une extension optionnelle du comportement de base plutôt qu'un cas d'utilisation systématique.

### 2.1.5. Règles de gestion

Les règles de gestion ci-dessous décrivent le comportement métier implémenté dans le code, tel que validé dans le dossier d'analyse. Chaque règle porte un numéro unique (RG-01 à RG-25), référencé tout au long de ce rapport dès qu'un écran, une table ou un extrait de code en dépend directement. Elles sont regroupées par thématique, dans l'ordre où un bénévole les rencontre concrètement : d'abord son compte, puis les événements auxquels il peut s'inscrire, le cycle de vie de son inscription, les avis qu'il peut laisser, son niveau de fidélité, ses attestations, et enfin les mécanismes transversaux de sécurité.

#### A. Comptes bénévoles

Ce premier groupe encadre la création et la gestion du compte : il garantit qu'un email identifie une personne unique, que le mot de passe ne peut jamais fuiter en clair, et qu'un compte problématique peut être neutralisé sans perdre l'historique associé — un point important pour Terra Sana, qui souhaite conserver une trace des participations passées même après le départ d'un bénévole.

**RG-01** — L'adresse email d'un bénévole est unique dans le système : elle sert d'identifiant de connexion et empêche la création de doublons.

**RG-02** — Le mot de passe compte au moins 8 caractères et est stocké haché (BCrypt) ; il n'est jamais consultable en clair.

**RG-03** — Un compte bénévole peut être désactivé sans être supprimé (isActive = false), afin de conserver l'historique des inscriptions passées.

**RG-04** — La demande de réinitialisation du mot de passe utilise un jeton à usage unique valable 30 minutes, envoyé par email.

#### B. Événements

Ce deuxième groupe fixe les données minimales qu'un événement doit porter pour être exploitable (date, lieu, capacité), la façon dont son statut évolue de lui-même au fil du temps, et une garde-fou pratique demandé directement par Monsieur Seraye après qu'un doublon de créneau se soit déjà produit dans les fichiers Excel de l'association.

**RG-05** — Chaque événement porte obligatoirement une date, un lieu et un nombre maximum de participants.

**RG-06** — Le statut d'un événement évolue automatiquement : OPEN par défaut, FULL dès que toutes les places sont prises, FINISHED une fois la date passée, CANCELLED uniquement sur décision explicite de l'administrateur.

**RG-07** — Un événement passé (FINISHED ou CANCELLED) n'accepte plus de nouvelles inscriptions.

**RG-08** — Deux événements ne peuvent pas être créés à la même date et au même lieu, indépendamment de leur titre, afin d'éviter les doublons accidentels.

### C. Inscriptions et liste d'attente

Ce troisième groupe est le plus dense, car il porte le cœur métier du module : garantir qu'une place ne soit jamais attribuée deux fois, offrir une liste d'attente lorsque la capacité est atteinte, et automatiser autant que possible la communication avec le bénévole à chaque étape, sans jamais retirer à l'administrateur le dernier mot sur une validation.

**RG-09** — Un bénévole ne peut s'inscrire qu'une seule fois au même événement (contrainte d'unicité sur le couple bénévole + événement).

**RG-10** — Si l'événement est complet au moment de l'inscription, le bénévole est placé automatiquement en liste d'attente (status = WAITING) avec une position calculée selon l'ordre d'arrivée.

**RG-11** — Toute inscription reste en attente de validation ; l'administrateur doit explicitement la confirmer ou la refuser.

**RG-12** — Un email de confirmation est envoyé automatiquement au bénévole lors de chaque validation ou refus.

**RG-13** — Lorsqu'une place se libère à la suite d'une désinscription confirmée, le premier bénévole en liste d'attente est promu automatiquement (WAITING → CONFIRMED) et notifié par email.

**RG-14** — Le bénévole peut se désinscrire librement tant que l'événement n'est pas commencé.

### D. Avis (retours post-événement)

Ce quatrième groupe encadre les retours laissés par les bénévoles : ils ne sont recevables que de la part de participants ayant réellement pris part à l'événement, et seulement une fois celui-ci terminé, afin d'éviter les avis anticipés ou de complaisance.

**RG-15** — Seul un bénévole ayant effectivement participé à un événement (inscription CONFIRMED) peut laisser un avis.

**RG-16** — Un avis ne peut être laissé qu'après la fin de l'événement (status = FINISHED).

**RG-17** — Un bénévole ne peut laisser qu'un seul avis par événement.

**RG-18** — La note est obligatoirement comprise entre 1 et 5.

### E. Niveau de fidélité

Ce cinquième groupe traduit un besoin explicitement formulé par Terra Sana lors des entretiens : reconnaître l'engagement des bénévoles les plus actifs sans mettre en place un système de points complexe. Le niveau reste un indicateur simple, recalculé automatiquement, sans intervention manuelle de l'administrateur.

**RG-19** — Le niveau du bénévole est calculé automatiquement à partir de son nombre de participations confirmées : BRONZE (1 à 2 événements), ARGENT (3 à 6), OR (7 et plus).

**RG-20** — Un email de notification est envoyé au bénévole lors du passage à un nouveau niveau.

### F. Attestations et exports

Ce sixième groupe répond directement à l'une des difficultés identifiées en introduction : l'absence de tout moyen, avant ce module, de délivrer une preuve de participation à un bénévole. L'attestation devient accessible en libre-service, sous réserve d'une participation réellement confirmée.

**RG-21** — Une attestation PDF de participation ne peut être générée que si le bénévole a au moins une participation confirmée à un événement terminé.

**RG-22** — L'export PDF de la liste des inscrits d'un événement est réservé à l'administrateur.

## G. Sécurité et accès

Ce dernier groupe fixe les garanties transversales de sécurité, applicables à l'ensemble du module plutôt qu'à un module fonctionnel en particulier : durée de vie des jetons, séparation stricte entre les droits d'un bénévole et ceux de l'administrateur.

**RG-23** — Toute action nécessitant un compte est protégée par un jeton JWT valable 24 heures ; l'expiration force la reconnexion.

**RG-24** — Les endpoints d'administration sont accessibles uniquement aux comptes disposant du rôle administrateur.

**RG-25** — Les tokens de bénévole et d'administrateur sont émis séparément, avec des secrets et des durées de vie distincts.

### 2.1.6. Comparaison avec le fonctionnement manuel antérieur

Pour mesurer concrètement l'apport du module, le tableau ci-dessous met en regard, tâche par tâche, la façon dont Terra Sana procédait avant ce TFE (section 1.3) et la façon dont la même tâche est désormais prise en charge automatiquement, avec la ou les règles de gestion qui la garantissent.

| Tâche                              | Avant (fichiers Excel / email)                                                        | Après (module TFE)                                                                    |
|------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Suivi des places disponibles       | Recomptage manuel des inscrits dans un fichier partagé, souvent désynchronisé         | Décompte automatique des inscriptions CONFIRMED, jamais stocké en dur (RG-06)         |
| Gestion du dépassement de capacité | Aucune règle : les événements étaient acceptés sans limite, ou refusés au cas par cas | Bascule automatique en liste d'attente dès que maxPlaces est atteint (RG-10)          |
| Confirmation d'une inscription     | Réponse individuelle par email, rédigée manuellement pour chaque bénévole             | Email automatique généré à chaque validation ou refus (RG-12)                         |
| Suivi d'un désistement             | Aucun mécanisme pour recontacter la liste d'attente : la place restait perdue         | Promotion automatique du premier bénévole en attente, avec notification (RG-13)       |
| Preuve de participation            | Inexistante : aucun document n'était délivré aux bénévoles                            | Attestation PDF téléchargeable en libre-service après participation confirmée (RG-21) |
| Reconnaissance de l'engagement     | Aucune, faute d'historique consolidé et exploitable                                   | Niveau de fidélité calculé automatiquement à partir de l'historique réel (RG-19)      |
| Détection des doublons de créneaux | Aucune, à l'origine d'un incident concret ayant motivé la règle RG-08                 | Rejet automatique à la création d'un événement au même lieu et à la même date         |

Tableau 4 — Comparaison entre le fonctionnement manuel antérieur et le module TFE

### 2.1.7. D'une règle de gestion à son implémentation : l'exemple de RG-08

Pour illustrer concrètement le passage d'une règle de gestion à son implémentation, cette section détaille RG-08, dont la version définitive résulte directement d'un contre-exemple rencontré en cours de développement (repris en section 4.1). Une première version comparait le titre et l'horaire exacts de deux événements pour détecter un doublon ; elle s'est révélée insuffisante face à deux événements portant des titres légèrement différents (« location des voitures » et « locations des voitures ») mais créés au même lieu, le même jour — un doublon réel que cette première version ne détectait pas. La méthode checkNoDuplicate

d'EventService, appelée aussi bien à la création qu'à la modification d'un événement, applique la version corrigée :

```
// Empêche de créer/déplacer un événement vers le même jour + le même lieu qu'un
// événement déjà existant, indépendamment du titre.
private void checkNoDuplicate(String location, LocalDateTime eventDate, Long excludeId) {
    LocalDateTime startOfDay = eventDate.toLocalDate().atStartOfDay();
    LocalDateTime endOfDay = startOfDay.plusDays(1);
    boolean duplicate = eventRepo
        .findByEventDateBetweenAndLocationIgnoreCase(startOfDay, endOfDay, location)
        .stream()
        .anyMatch(e -> !e.getId().equals(excludeId));
    if (duplicate) {
        throw new RuntimeException("Un événement existe déjà le "
            + eventDate.format(DUPLICATE_MSG_FMT) + " à " + location + ".");
    }
}
```

Deux détails techniques méritent d'être relevés. D'abord, la comparaison se fait sur une plage de la journée entière (startOfDay à endOfDay) plutôt que sur l'horaire exact, afin que deux événements à 9h et à 15h le même jour et au même lieu soient bien détectés comme un doublon. Ensuite, le paramètre excludeId permet à la méthode d'être appelée aussi bien à la création (excludeId = null, aucun événement à exclure) qu'à la modification (excludeId = l'identifiant de l'événement en cours d'édition, afin qu'un événement ne se voie pas signalé comme son propre doublon lorsqu'on modifie simplement sa description sans changer ni la date ni le lieu).

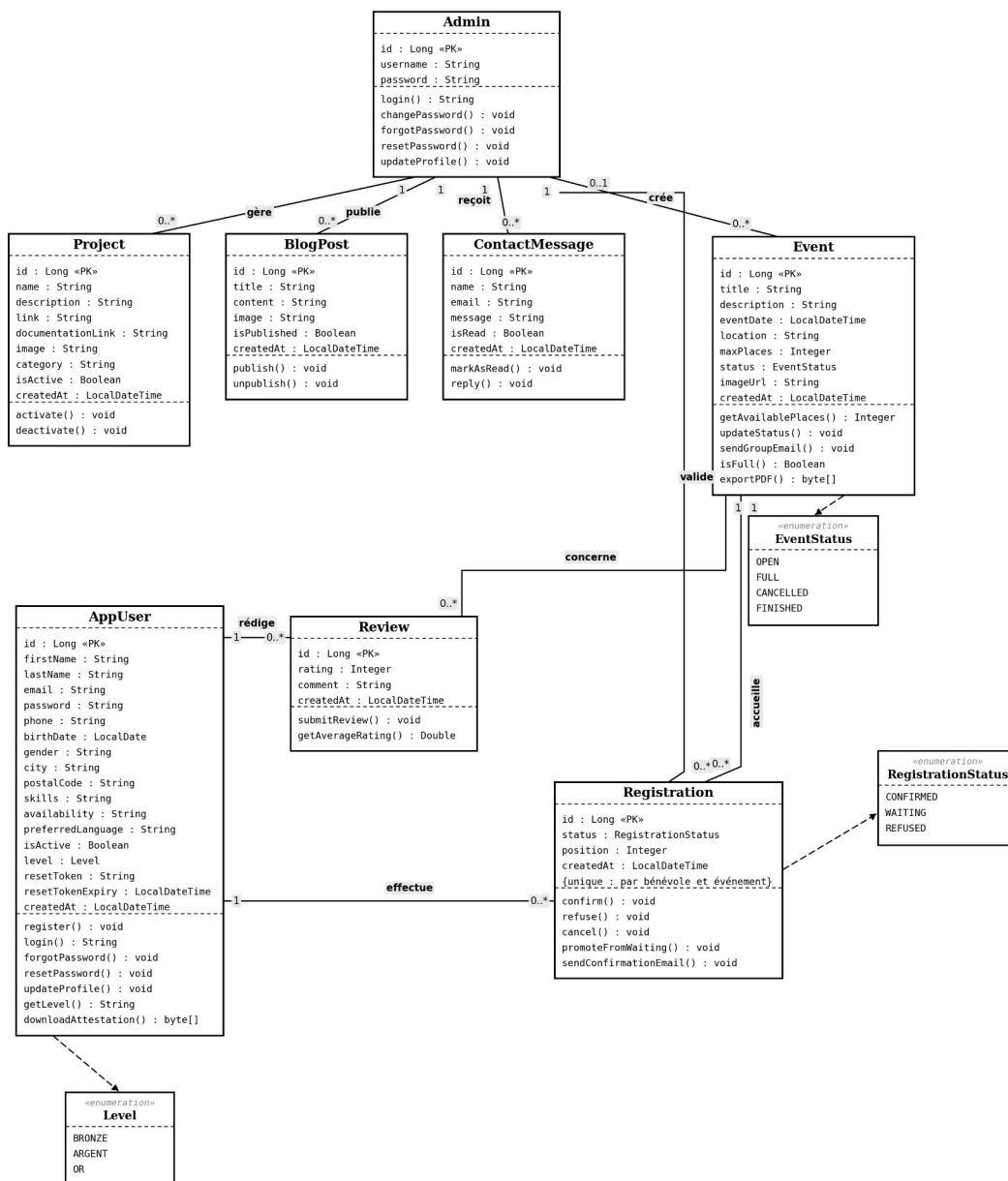
## 2.2. Persistance des données

Le module interagit avec une base de données relationnelle MySQL. Cette section présente les trois livrables demandés par les consignes : le diagramme de classes, le dictionnaire de données qui le documente, puis le schéma physique de la base.

### 2.2.1. Diagramme de classes

Le diagramme de classes ci-dessous modélise la structure statique de l'application : huit entités, leurs attributs, leurs méthodes et les associations qui les relient. Conformément aux consignes, les clés étrangères ne sont pas dupliquées comme attributs dans les classes : seules les associations nommées, portées par les liens entre classes, expriment ces relations. S'agissant d'un sujet personnel construit après le stage, ce diagramme a été soumis à Madame Marie-Christine Namur et validé par elle avant le début du développement ; il n'a plus été modifié depuis, conformément à cette validation.

Admin, Project, BlogPost et ContactMessage proviennent du site existant, livré durant le stage. AppUser, Event, Registration et Review sont les quatre entités créées spécifiquement pour ce TFE. Trois énumérations (EventStatus, Level, RegistrationStatus) typent les attributs à valeurs fermées et sont reliées par une dépendance en trait pointillé aux classes qui les utilisent.



Les neuf associations du diagramme se lisent comme suit : un Admin gère plusieurs Project, publie plusieurs BlogPost, reçoit plusieurs ContactMessage et crée plusieurs Event (relations 1 — 0..\*); un Admin valide 0 ou plusieurs Registration (relation 0..1 — 0..\* côté Registration, puisqu'une inscription n'a pas encore d'administrateur validateur tant qu'elle est en attente) ; un AppUser rédige plusieurs Review et effectue plusieurs Registration ; enfin, un Event accueille plusieurs Registration et concerne plusieurs Review.

## 2.2.2. Dictionnaire de données

Le dictionnaire ci-dessous reprend l'ensemble des tables de la base MySQL terresana, à la fois celles héritées du stage et les quatre nouvelles tables créées pour le module TFE. Conformément aux consignes, il précise, pour chaque champ, le type SQL, les contraintes et le rôle métier — sans mentionner les clés étrangères, déjà exprimées par les associations nommées du diagramme de classes (section 2.2.1) ; elles ne réapparaissent

qu'au niveau du schéma physique (section 2.2.3), seul niveau où la clé étrangère est une réalité technique et non plus une redondance.

### Tables héritées du stage

| Champ    | Type SQL     | Contrainte         | Description                  |
|----------|--------------|--------------------|------------------------------|
| id       | BIGINT       | PK, AUTO_INCREMENT | Identifiant.                 |
| username | VARCHAR(100) | NOT NULL, UNIQUE   | Identifiant de connexion.    |
| password | VARCHAR(255) | NOT NULL           | Mot de passe haché (BCrypt). |

*Table admin — compte administrateur unique*

| Champ                    | Type SQL     | Contrainte         | Description                |
|--------------------------|--------------|--------------------|----------------------------|
| id                       | BIGINT       | PK, AUTO_INCREMENT | Identifiant.               |
| name                     | VARCHAR(200) | NOT NULL           | Nom de l'application.      |
| description              | TEXT         | NOT NULL           | Présentation.              |
| link, documentation_link | VARCHAR(500) | NULL               | URL vers l'app et sa doc.  |
| image, category          | VARCHAR(255) | NULL               | Illustration et catégorie. |
| is_active                | BOOLEAN      | DEFAULT TRUE       | Visible ou non sur le hub. |
| created_at               | DATETIME     | NOT NULL           | Date de création.          |

*Table project — applications du hub interne*

| Champ          | Type SQL       | Contrainte         | Description           |
|----------------|----------------|--------------------|-----------------------|
| id             | BIGINT         | PK, AUTO_INCREMENT | Identifiant.          |
| title, content | VARCHAR / TEXT | NOT NULL           | Contenu de l'article. |
| image          | VARCHAR(255)   | NULL               | Illustration.         |
| is_published   | BOOLEAN        | DEFAULT FALSE      | Publié ou brouillon.  |
| created_at     | DATETIME       | NOT NULL           | Date de rédaction.    |

*Table blog\_post — articles du blog*

| Champ       | Type SQL      | Contrainte         | Description                  |
|-------------|---------------|--------------------|------------------------------|
| id          | BIGINT        | PK, AUTO_INCREMENT | Identifiant.                 |
| name, email | VARCHAR(255)  | NOT NULL           | Coordonnées de l'expéditeur. |
| message     | VARCHAR(5000) | NOT NULL           | Contenu du message.          |
| is_read     | BOOLEAN       | DEFAULT FALSE      | Traité ou non.               |
| created_at  | DATETIME      | NOT NULL           | Date de réception.           |

*Table contact\_message — messages du formulaire de contact*

### Nouvelles tables du module TFE

app\_users porte l'ensemble du profil bénévole. Seuls quatre champs sont obligatoires (nom, prénom, email, mot de passe) ; tous les autres — téléphone, ville, compétences, disponibilités — restent facultatifs, conformément au principe de minimisation des données appliqué sur le formulaire d'inscription (section 3.4.2). Le champ level est dénormalisé à dessein : plutôt que de recalculer le niveau à chaque affichage à

partir de l'historique complet des inscriptions, il est stocké et mis à jour explicitement par `updateLevel()` (section 3.3.3), ce qui évite de recompter les participations confirmées à chaque chargement du profil.

| Champ                     | Type SQL          | Contrainte            | Description                        |
|---------------------------|-------------------|-----------------------|------------------------------------|
| id                        | BIGINT            | PK,<br>AUTO_INCREMENT | Identifiant.                       |
| first_name, last_name     | VARCHAR(100)      | NOT NULL              | Identité du bénévole.              |
| email                     | VARCHAR(200)      | NOT NULL, UNIQUE      | Email de connexion (RG-01).        |
| password                  | VARCHAR(255)      | NOT NULL              | Haché BCrypt (RG-02).              |
| phone, birth_date, gender | VARCHAR /<br>DATE | NULL                  | Coordonnées facultatives.          |
| city, postal_code         | VARCHAR(255)      | NULL                  | Localité.                          |
| skills, availability      | TEXT /<br>VARCHAR | NULL                  | Compétences et disponibilités.     |
| preferred_language        | VARCHAR(5)        | DEFAULT 'fr'          | Langue préférée (FR/EN/NL).        |
| level                     | VARCHAR(10)       | DEFAULT 'BRONZE'      | BRONZE / ARGENT / OR (RG-19).      |
| is_active                 | BOOLEAN           | DEFAULT TRUE          | Compte actif (RG-03).              |
| reset_token               | VARCHAR(255)      | NULL                  | Jeton de réinitialisation (RG-04). |
| reset_token_expiry        | DATETIME          | NULL                  | Expiration du jeton.               |
| created_at                | DATETIME          | NOT NULL              | Date d'inscription.                |

*Table app\_users — bénévoles*

events porte les données propres à chaque activité organisée par Terra Sana, ainsi qu'une référence vers l'administrateur qui l'a créée — utile dès lors que Terra Sana envisagerait, à l'avenir, plusieurs comptes administrateurs (section 4.4). Le champ status n'est pas laissé au libre choix de l'administrateur : seule la valeur CANCELLED lui est réellement ouverte, OPEN/FULL/FINISHED étant recalculés par EventService selon le remplissage et la date (RG-06).

| Champ       | Type SQL     | Contrainte            | Description                           |
|-------------|--------------|-----------------------|---------------------------------------|
| id          | BIGINT       | PK,<br>AUTO_INCREMENT | Identifiant.                          |
| title       | VARCHAR(255) | NOT NULL              | Titre.                                |
| description | TEXT         | NOT NULL              | Description complète.                 |
| event_date  | DATETIME     | NOT NULL              | Date et heure (RG-05).                |
| location    | VARCHAR(300) | NOT NULL              | Lieu.                                 |
| max_places  | INT          | NOT NULL              | Nombre de places (RG-05).             |
| status      | VARCHAR(20)  | DEFAULT 'OPEN'        | OPEN/FULL/CANCELLED/FINISHED (RG-06). |
| image_url   | VARCHAR(500) | NULL                  | Illustration.                         |
| created_at  | DATETIME     | NOT NULL              | Date de création.                     |

*Table events — événements*

registrations est la table pivot du module, associant un bénévole et un événement, et retraçant également quel administrateur a traité la demande. Tant que l'administrateur n'a pas explicitement statué (RG-11), cette information de traitement reste vide, ce qui permet de distinguer une inscription encore en attente d'une

inscription refusée sans ambiguïté. La contrainte d'unicité porte à elle seule toute la garantie de RG-09, indépendamment de ce que vérifie ou non le code applicatif au moment de l'insertion.

| Champ      | Type SQL    | Contrainte                     | Description                                                               |
|------------|-------------|--------------------------------|---------------------------------------------------------------------------|
| id         | BIGINT      | PK,<br>AUTO_INCREMENT          | Identifiant.                                                              |
| status     | VARCHAR(20) | NOT NULL, DEFAULT<br>'WAITING' | CONFIRMED / WAITING / REFUSED.                                            |
| position   | INT         | NULL                           | Position en liste (RG-10).                                                |
| created_at | DATETIME    | NOT NULL                       | Date d'inscription.                                                       |
| —          | —           | Unicité                        | Un bénévole ne peut avoir qu'une seule inscription par événement (RG-09). |

*Table registrations — inscriptions et liste d'attente*

reviews associe, comme registrations, un bénévole (RG-15) et un événement, avec la même contrainte d'unicité, mais répond à un besoin différent : elle ne trace pas une participation, mais un avis, et n'existe que pour les événements réellement terminés. La contrainte CHECK sur rating traduit directement RG-18 au niveau base de données, en complément — et non à la place — de la validation déjà effectuée côté frontend et côté service. La note moyenne affichée sur le tableau de bord (section 3.4.6) n'est, comme les places disponibles d'un événement, jamais stockée : elle est recalculée à chaque consultation à partir de l'ensemble des avis de l'événement — `reviews.stream().mapToInt(Review::getRating).average()` — plutôt que maintenue comme un total glissant qui risquerait de se désynchroniser des avis réellement en base.

| Champ      | Type SQL | Contrainte              | Description                                                        |
|------------|----------|-------------------------|--------------------------------------------------------------------|
| id         | BIGINT   | PK,<br>AUTO_INCREMENT   | Identifiant.                                                       |
| rating     | INT      | NOT NULL, CHECK<br>1..5 | Note (RG-18).                                                      |
| comment    | TEXT     | NULL                    | Commentaire libre.                                                 |
| created_at | DATETIME | NOT NULL                | Date de l'avis.                                                    |
| —          | —        | Unicité                 | Un bénévole ne peut laisser qu'un seul avis par événement (RG-17). |

*Table reviews — retours post-événement*

### 2.2.3. Schéma physique de la base de données

Le schéma physique découle directement du diagramme de classes et du dictionnaire de données ci-dessus : chaque entité devient une table, chaque association devient une clé étrangère avec une contrainte référentielle. La base respecte les règles de normalisation (3NF). Le script SQL complet (création des huit tables, contraintes et index) est reproduit intégralement en Annexe A ; un extrait représentatif — la création des tables events et registrations, qui concentrent l'essentiel des règles de gestion — est présenté ci-dessous.

```
CREATE TABLE events (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  admin_id BIGINT, title VARCHAR(255) NOT NULL,
  description TEXT NOT NULL, event_date DATETIME NOT NULL,
  location VARCHAR(300) NOT NULL, max_places INT NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'OPEN', image_url VARCHAR(500),
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_event_admin FOREIGN KEY (admin_id) REFERENCES admin(id)
);

CREATE TABLE registrations (
```

```

id BIGINT AUTO_INCREMENT PRIMARY KEY,
user_id BIGINT NOT NULL, event_id BIGINT NOT NULL, validated_by_id BIGINT,
status VARCHAR(20) NOT NULL DEFAULT 'WAITING', position INT,
created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT uq_user_event UNIQUE (user_id, event_id),
CONSTRAINT fk_reg_user FOREIGN KEY (user_id) REFERENCES app_users(id) ON DELETE CASCADE,
CONSTRAINT fk_reg_event FOREIGN KEY (event_id) REFERENCES events(id) ON DELETE CASCADE,
CONSTRAINT fk_reg_admin FOREIGN KEY (validated_by_id) REFERENCES admin(id)
);

```

La normalisation en troisième forme normale se vérifie concrètement sur chaque table : aucun champ n'est une donnée calculable à partir d'autres champs de la même ligne (le nombre de places disponibles d'un événement, par exemple, n'est jamais stocké — il est recalculé à la demande à partir du décompte des inscriptions CONFIRMED, évitant tout risque d'incohérence entre une valeur stockée et la réalité des inscriptions), et chaque attribut non-clé dépend de la clé primaire entière plutôt que d'une partie de celle-ci. La contrainte UNIQUE(user\_id, event\_id) sur registrations et reviews illustre ce même souci : plutôt que de vérifier applicativement qu'un bénévole ne s'inscrit pas deux fois (RG-09) ou n'avise pas deux fois (RG-17), la contrainte est portée par la base elle-même, qui rejette toute tentative d'insertion en doublon indépendamment du code applicatif qui l'a produite.

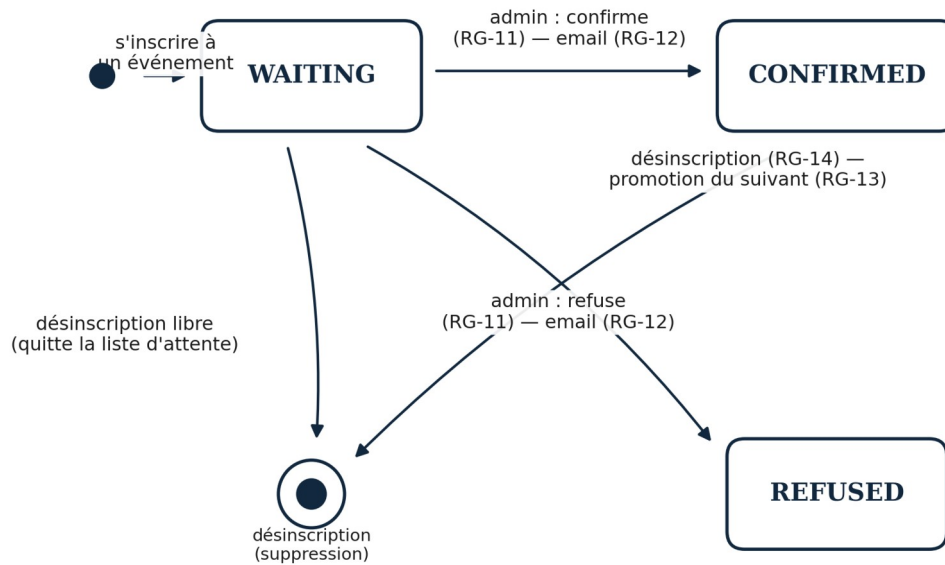
Deux stratégies de suppression en cascade sont appliquées de façon différenciée : ON DELETE CASCADE sur les clés étrangères vers app\_users et events (la suppression d'un bénévole ou d'un événement entraîne logiquement celle de ses inscriptions et avis), mais aucune suppression en cascade n'est définie vers admin, précisément parce que RG-03 impose qu'un compte bénévole soit désactivé plutôt que supprimé — la cascade ne s'applique donc en pratique jamais dans ce sens pour les données de bénévoles actifs.

## 2.3. Autres diagrammes UML

Cette section ajoute deux diagrammes d'état UML, chacun modélisant le cycle de vie complet d'une entité du module à travers ses transitions possibles — le pendant dynamique du diagramme de classes, qui ne décrit que la structure statique. Chaque diagramme est accompagné d'un tableau détaillant, transition par transition, la règle de gestion qui la déclenche.

Le premier retrace le cycle de vie d'une inscription (Registration), directement gouverné par les règles RG-09 à RG-13 : toute inscription naît WAITING, qu'il y ait ou non une place disponible, puis évolue vers CONFIRMED ou REFUSED sur décision explicite de l'administrateur. Une désinscription confirmée déclenche la promotion automatique du premier bénévole en liste d'attente ; le pseudo-état final (cercle plein cerclé) représente la suppression de la ligne d'inscription elle-même, qu'elle intervienne depuis WAITING (désinscription libre) ou depuis CONFIRMED (désinscription avec promotion du suivant).

### Cycle de vie d'une inscription (Registration)



| Étape | Transition                 | Règle de gestion                                                                                     |
|-------|----------------------------|------------------------------------------------------------------------------------------------------|
| 1     | Création → WAITING         | Toute inscription démarre en attente, avec position calculée si l'événement est déjà complet (RG-10) |
| 2     | WAITING → CONFIRMED        | Validation explicite par l'administrateur (RG-11) ; email de confirmation envoyé (RG-12)             |
| 3     | WAITING → REFUSED          | Refus explicite par l'administrateur (RG-11) ; email envoyé (RG-12)                                  |
| 4     | CONFIRMED → (suppression)  | Désinscription volontaire du bénévole, possible avant le début de l'événement (RG-14)                |
| 5     | WAITING → CONFIRMED (auto) | Promotion automatique du premier en liste d'attente après une désinscription confirmée (RG-13)       |

Tableau 5 — Détail des transitions du cycle de vie d'une inscription

La transition WAITING → CONFIRMED illustrée ci-dessus n'est pas une simple mise à jour de statut : `RegistrationService.confirm()` revérifie, au moment même de la confirmation, que l'événement n'est pas déjà complet, plutôt que de faire confiance à l'état affiché côté frontend au moment où l'administrateur a cliqué :

```

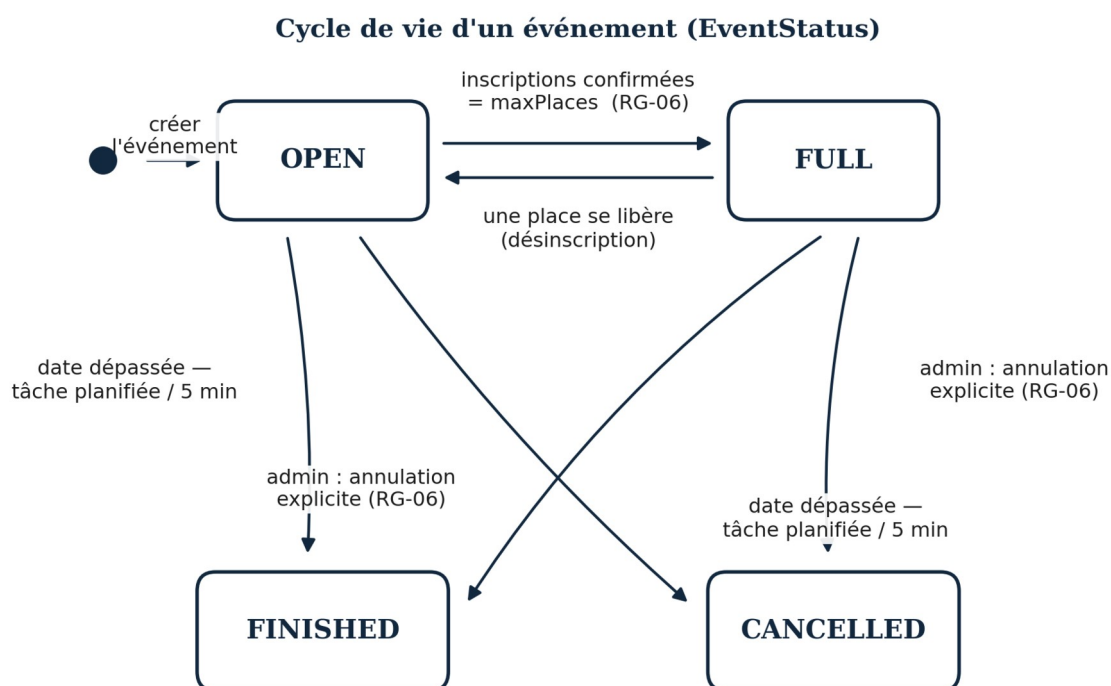
public Registration confirm(Long registrationId, Admin admin) {
    Registration reg = findById(registrationId);
    long alreadyConfirmed = registrationRepo
        .countByEventAndStatus(reg.getEvent(), RegistrationStatus.CONFIRMED);
    if (alreadyConfirmed >= reg.getEvent().getMaxPlaces()) {
        throw new RuntimeException("Cet événement est déjà complet, " +
            "impossible de confirmer une inscription supplémentaire.");
    }
    reg.setStatus(RegistrationStatus.CONFIRMED);
    reg.setPosition(null);
    reg.setValidatedBy(admin);
    return registrationRepo.save(reg);
}

```

Ce recomptage au moment de l'action, plutôt qu'une simple lecture d'un compteur potentiellement obsolète, évite un cas limite réel : si l'administrateur laisse deux onglets de son tableau de bord ouverts et confirme

depuis chacun une inscription en attente pour le dernier créneau disponible, seule la première confirmation réussit — la seconde échoue proprement avec un message explicite plutôt que de dépasser silencieusement la capacité de l'événement. La désinscription applique une distinction symétrique : quitter la liste d'attente ne libère aucune place, seule l'annulation d'une inscription CONFIRMED déclenche la recherche d'un remplaçant (RG-13).

Le second diagramme retrace le cycle de vie d'un événement (Event), qui suit une logique différente de celle d'une inscription : OPEN et FULL sont réversibles l'un vers l'autre selon le remplissage réel (checkAndMarkFull / checkAndMarkOpen, section 3.2.3), tandis que FINISHED et CANCELLED sont tous deux définitifs — le premier atteint automatiquement par la tâche planifiée dès que la date est dépassée, le second uniquement sur décision explicite de l'administrateur, à tout moment de la vie de l'événement.



| État      | Condition                                                                                                 | Règle de gestion |
|-----------|-----------------------------------------------------------------------------------------------------------|------------------|
| OPEN      | Statut par défaut à la création, tant que des places restent disponibles                                  | RG-06            |
| FULL      | Le nombre d'inscriptions CONFIRMED atteint maxPlaces                                                      | RG-06            |
| FINISHED  | La date de l'événement (eventDate) est dépassée — bascule automatique par tâche planifiée (section 3.2.3) | RG-06, RG-07     |
| CANCELLED | Uniquement sur décision explicite de l'administrateur, à tout moment                                      | RG-06            |

Tableau 6 — Détail des transitions du cycle de vie d'un événement

Un événement CANCELLED ou FINISHED n'accepte plus de nouvelle inscription (RG-07) : le bouton « S'inscrire » disparaît alors de l'interface publique, et le contrôleur rejette également toute tentative d'inscription envoyée directement à l'API pour ce statut, la vérification n'étant jamais laissée à la seule charge du frontend.

En complément, les deux diagrammes de séquence ci-dessous détaillent, étape par étape, les interactions entre le frontend React, les contrôleurs Spring Boot et la base de données pour les deux scénarios les plus représentatifs du module : l'inscription à un événement, et la promotion automatique depuis la liste d'attente.

### Scénario 1 — Inscription d'un bénévole à un événement

| Étape | Acteur                 | Action                                                                                                           |
|-------|------------------------|------------------------------------------------------------------------------------------------------------------|
| 1     | Bénévole               | Consulte la liste des événements et clique sur « S'inscrire » pour un événement donné                            |
| 2     | Frontend (React)       | Envoie POST /api/registrations/event/{id} avec le jeton JWT du bénévole dans l'en-tête Authorization             |
| 3     | JwtFilter              | Valide le jeton avec VolunteerJwtUtil, extrait l'identifiant du bénévole et l'assigne ROLE_VOLUNTEER             |
| 4     | RegistrationController | Reçoit la requête authentifiée et délègue au RegistrationService                                                 |
| 5     | RegistrationService    | Vérifie qu'aucune inscription n'existe déjà pour ce couple bénévole/événement (RG-09)                            |
| 6     | RegistrationService    | Compare le nombre d'inscriptions CONFIRMED au maxPlaces de l'événement pour déterminer le statut initial (RG-10) |
| 7     | RegistrationRepository | Persiste la nouvelle inscription (INSERT), avec position calculée si le statut est WAITING                       |
| 8     | RegistrationController | Retourne l'inscription créée (200 OK)                                                                            |
| 9     | Frontend (React)       | Met à jour l'affichage : confirmation immédiate, ou message indiquant la position en liste d'attente             |

Tableau 7 — Diagramme de séquence (détail des étapes) : inscription à un événement

### Scénario 2 — Promotion automatique depuis la liste d'attente

| Étape | Acteur                 | Action                                                                                                 |
|-------|------------------------|--------------------------------------------------------------------------------------------------------|
| 1     | Administrateur         | Confirme la désinscription d'un bénévole déjà CONFIRMED pour un événement                              |
| 2     | Frontend (React)       | Envoie PUT /api/registrations/{id}/cancel avec le jeton JWT de l'administrateur                        |
| 3     | RegistrationController | Reçoit la requête et délègue au RegistrationService                                                    |
| 4     | RegistrationService    | Supprime l'inscription confirmée, libérant une place sur l'événement (RG-14)                           |
| 5     | RegistrationService    | Recherche, parmi les inscriptions WAITING du même événement, celle dont la position est la plus faible |
| 6     | RegistrationService    | Fait passer cette inscription de WAITING à CONFIRMED (RG-13)                                           |
| 7     | EmailService           | Envoie automatiquement un email de confirmation au bénévole promu (RG-12)                              |
| 8     | RegistrationRepository | Persiste les deux modifications (suppression puis mise à jour du statut)                               |
| 9     | Frontend (React)       | Rafraîchit la liste des inscrits et la liste d'attente affichées à l'administrateur                    |

Tableau 8 — Diagramme de séquence (détail des étapes) : promotion automatique depuis la liste d'attente

## III. Dossier technique

Cette section décrit les décisions d'implémentation du module : les choix technologiques et leur justification, l'architecture globale, les principaux écrans avec leur fonctionnement, la sécurité mise en place et l'environnement de développement retenu.

### 3.1. Choix technologiques

La pile technique du module TFE s'appuie sur les technologies enseignées dans le cursus (Java, Spring Boot, MySQL), en complément de React déjà utilisé côté frontend depuis le stage. Elle diffère volontairement de certaines stacks JavaScript full-stack plus légères, afin de démontrer la maîtrise d'une architecture Java d'entreprise typique.

| Composant                 | Technologie                           | Justification                                                                               |
|---------------------------|---------------------------------------|---------------------------------------------------------------------------------------------|
| Framework backend         | Spring Boot 3.5.13 (Java 21)          | Auto-configuration, large écosystème, standard de l'industrie pour les API d'entreprise     |
| Accès aux données         | Spring Data JPA / Hibernate           | Abstraction du SQL, requêtes dérivées par convention de nommage, mapping objet-relationnel  |
| Sécurité                  | Spring Security + jjwt 0.11.5         | Authentification stateless par JWT, adaptée à une SPA React ; deux jetons distincts (RG-25) |
| Hachage des mots de passe | BCrypt (Spring Security)              | Sel automatique, résistant aux attaques par table arc-en-ciel (RG-02)                       |
| Base de données           | MySQL 9.1.0                           | SGBD relationnel enseigné en cours d'analyse, bonnes pratiques de normalisation             |
| Envoi d'e-mails           | Spring Boot Starter Mail (Gmail SMTP) | Confirmations, relances et notifications automatiques (RG-12, RG-13, RG-20)                 |
| Génération de PDF         | iText7 7.2.5                          | Attestations bénévole et export de la liste des inscrits (RG-21, RG-22)                     |
| Frontend                  | React 18.2.0                          | Continuité avec le site développé pendant le stage, composants réutilisables                |
| Graphiques                | Chart.js                              | Courbe des inscriptions par mois, répartition des statuts sur le tableau de bord            |
| Environnement local       | WAMP Server 3.3.7                     | Serveur MySQL local sous Windows pour le développement et la démonstration                  |

Tableau 9 — Choix technologiques du module TFE

La figure ci-dessous illustre cette pile technique par grande couche, chaque bloc étant coloré selon l'identité visuelle usuelle de la technologie qu'il représente.

#### BACKEND



#### FRONTEND



#### DONNÉES & SÉCURITÉ



#### DOCUMENTS & OUTILS



Deux alternatives ont été écartées consciemment. Côté base de données, une solution NoSQL (MongoDB) aurait été envisageable pour sa flexibilité de schéma, mais les données du module (bénévoles, événements, inscriptions) sont fortement relationnelles — une inscription n'a de sens qu'accompagnée d'un bénévole ET d'un événement existants, avec une contrainte d'unicité stricte sur ce couple (RG-09) — ce que MySQL exprime nativement via des clés étrangères et une contrainte UNIQUE, là où une base documentaire demanderait de réimplémenter ces garanties manuellement dans le code applicatif. Côté backend, une pile Node.js/Express aurait permis de rester dans un unique langage avec le frontend React, mais Spring Boot a été préféré pour rester cohérent avec les technologies du cursus académique et démontrer une architecture typée de bout en bout, du contrôleur jusqu'à la colonne SQL.

Le choix de Spring Data JPA plutôt que du JDBC pur ou d'un query builder plus léger se justifie par le volume de requêtes dérivées qu'autorise le module (section 3.2.2) : la majorité des accès aux données du module TFE s'exprime par de simples signatures de méthode, sans SQL manuel, ce qui réduit le risque d'erreur de syntaxe et concentre l'effort de développement sur la logique métier plutôt que sur l'écriture répétitive de requêtes. jjwt a été préféré à une implémentation JWT maison pour la même raison qui justifie BCrypt plutôt qu'un hachage SHA fait main : la cryptographie appliquée ne se réinvente pas, elle s'appuie sur des bibliothèques éprouvées et maintenues. Enfin, iText7 a été retenu pour la génération de PDF plutôt que JasperReports, envisagé un temps : pour des documents au contenu simple (une attestation, une liste tabulaire), iText7 permet un contrôle direct et minimal du contenu généré, sans la couche de templates XML que JasperReports impose pour des rapports plus complexes que ceux réellement nécessaires ici.

### 3.2. Architecture globale

Le module repose sur une architecture 3-tiers classique, séparant la couche de présentation (frontend React), la couche métier (API Spring Boot) et la couche de données (MySQL). Chaque module fonctionnel du backend (bénévoles, événements, inscriptions, avis) suit le pattern en couches standard de Spring Boot : un Controller reçoit la requête HTTP, un Service porte la logique métier, un Repository accède aux données via Spring Data JPA.

| Couche            | Classe type                                          | Responsabilité                                                          |
|-------------------|------------------------------------------------------|-------------------------------------------------------------------------|
| Controller        | EventController, RegistrationController              | Reçoit les requêtes HTTP, délègue au service, retourne la réponse JSON  |
| Service           | EventService, RegistrationService                    | Logique métier : règles de gestion, calcul de statut, envoi d'emails    |
| Repository        | EventRepository, RegistrationRepository              | Interfaces Spring Data JPA, requêtes dérivées et JPQL                   |
| Model             | Event, Registration, AppUser, Review                 | Entités JPA mappées sur les tables MySQL                                |
| Config / Security | JwtFilter, JwtUtil, VolunteerJwtUtil, SecurityConfig | Génération et validation des jetons, règles d'autorisation par endpoint |

Tableau 10 — Structure en couches du backend Spring Boot

La structure des packages reflète cette organisation :

```
com.terresana.backend
├── controller/   AuthController, AppUserController, EventController,
                  RegistrationController, ReviewController,
                  ProjectController, BlogPostController, ContactController
├── service/      AppUserService, EventService, RegistrationService,
                  ReviewService, EmailService, PdfService,
                  EventImageService, ProjectImageService
├── repository/   AppUserRepository, EventRepository,
                  RegistrationRepository, ReviewRepository, ...
├── model/        Admin, AppUser, Event, Registration, Review, ...
│   └── enums/    EventStatus, RegistrationStatus, Level
├── config/       SecurityConfig, JwtFilter, JwtUtil,
                  VolunteerJwtUtil, GlobalExceptionHandler
└── BackendApplication.java
```

Côté frontend, chaque page (Events.js, VolunteerDashboard.js, Admin.js...) consomme l'API REST via un service HTTP dédié (services/volunteerApi.js pour l'espace bénévole, services/api.js pour le site vitrine), et le token JWT courant est géré centralement dans services/auth.js.

### 3.2.1. Contrôle de la sérialisation JSON

L'entité AppUser illustre un point d'attention systématique du module : ne jamais laisser une information sensible transiter par erreur dans une réponse API, même de façon indirecte. Deux annotations Jackson y sont utilisées à dessein plutôt qu'une seule, pour deux besoins différents :

```
@JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
private String password; // acceptable en entrée (inscription), jamais renvoyé

@JsonIgnore
private String resetToken; // ni en entrée, ni en sortie : jamais exposé au client
```

Le mot de passe utilise WRITE\_ONLY et non @JsonIgnore, car @JsonIgnore aurait aussi bloqué la désérialisation du champ envoyé par le formulaire d'inscription — le mot de passe doit rester acceptable en entrée tout en étant systématiquement omis des réponses JSON. Le jeton de réinitialisation, à l'inverse, n'a jamais besoin d'être envoyé par le client : @JsonIgnore l'exclut complètement, empêchant qu'il puisse fuiter dans une réponse API et permettre une prise de compte.

### 3.2.2. Requêtes dérivées Spring Data JPA

Spring Data JPA permet de définir une requête simplement en nommant la méthode selon une convention précise, sans écrire la moindre ligne de SQL ou de JPQL — Spring génère l'implémentation à partir du nom lui-même. RegistrationRepository en est une illustration directe, chaque méthode correspondant à un besoin métier précis du module :

```
public interface RegistrationRepository extends JpaRepository<Registration, Long> {

    // Vérifier si un bénévole est déjà inscrit à un événement (RG-09)
    Optional<Registration> findByUserAndEvent(AppUser user, Event event);

    // Compter les places prises (CONFIRMED) pour savoir si l'événement est plein
    long countByEventAndStatus(Event event, RegistrationStatus status);

    // Liste d'attente triée par date d'inscription (le plus ancien passe en premier)
    List<Registration> findByEventAndStatusOrderByCreatedAtAsc(Event event, RegistrationStatus status);
}
```

Le nom `findByEventAndStatusOrderByCreatedAtAsc` se lit littéralement comme la règle qu'il implémente : trouver les inscriptions d'un événement (By Event), dont le statut correspond (And Status), triées par date de création croissante (OrderBy CreatedAt Asc) — c'est-à-dire exactement la liste d'attente dans l'ordre d'arrivée nécessaire à RG-13, sans qu'une seule ligne de logique de tri n'ait été écrite manuellement.

### 3.2.3. Intégrations externes et traitement planifié

Le module n'appelle aucune API externe à l'exécution : les photos associées aux événements (section 3.3.2) proviennent d'une table de correspondance fixe d'URL Unsplash déjà vérifiées, choisies localement par mots-clés, et non d'un appel réseau à un service tiers — un choix qui évite toute dépendance de disponibilité envers un service externe pour une fonctionnalité aussi secondaire qu'une image d'illustration. Le seul service externe réellement sollicité est le serveur SMTP de Gmail, pour l'envoi des e-mails (section 3.3.5).

Le module comporte en revanche un traitement planifié : une tâche `@Scheduled` de Spring, exécutée automatiquement toutes les cinq minutes, fait basculer en `FINISHED` tout événement encore marqué `OPEN` ou `FULL` dont la date est désormais dépassée (RG-06). Sans ce mécanisme, un événement resté `OPEN` après son déroulement continuerait, par exemple, à accepter de nouvelles inscriptions ou à autoriser un avis prématuré, en contradiction avec RG-07 et RG-16.

```
// Bascule automatiquement en FINISHED tout événement OPEN/FULL dont la date est
// passée. Tourne toutes les 5 minutes ; CANCELLED n'est jamais touché (décision
// explicite de l'administrateur uniquement).
@Scheduled(fixedRate = 300000)
public void autoMarkFinishedEvents() {
    List<Event> toFinish = eventRepo.findByEventDateBeforeAndStatusIn(
        LocalDateTime.now(), List.of(EventStatus.OPEN, EventStatus.FULL));
    if (!toFinish.isEmpty()) {
        toFinish.forEach(e -> e.setStatus(EventStatus.FINISHED));
        eventRepo.saveAll(toFinish);
    }
}
```

Le choix d'un intervalle de cinq minutes, plutôt qu'un calcul à la demande à chaque affichage, tient à la sémantique même du statut `FINISHED` : contrairement au nombre de places disponibles (section 2.2.3), qui doit toujours refléter l'instant présent, le passage à `FINISHED` déclenche des effets de bord — ouverture du dépôt d'avis (RG-16), éligibilité à l'attestation (RG-21) — qu'il est plus sûr de matérialiser explicitement en base à intervalles réguliers plutôt que de recalculer silencieusement à chaque lecture.

## 3.3. Fonctionnalités transversales notables

Quatre mécanismes, moins visibles qu'un écran mais représentatifs des choix d'implémentation retenus, méritent d'être détaillés : la gestion centralisée des erreurs, l'attribution automatique d'une photo aux événements et projets, l'envoi asynchrone des e-mails, et l'architecture retenue pour l'interface multilingue.

### 3.3.1. Gestion centralisée des erreurs (GlobalExceptionHandler)

Sans mécanisme dédié, une erreur métier levée dans un service Spring (email déjà utilisé, mot de passe incorrect, inscription déjà existante...) se traduit par défaut par une erreur 500 générique côté client, sans que le message clair écrit dans le service n'atteigne jamais le frontend. GlobalExceptionHandler intercepte, au niveau de l'application entière, toute RuntimeException levée par un contrôleur ou un service, et la transforme systématiquement en réponse HTTP 400 structurée :

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(RuntimeException.class)
    public ResponseEntity<Map<String, String>> handle(RuntimeException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
            .body(Map.of("message", e.getMessage()));
    }
}
```

Ce choix centralise en un seul endroit une responsabilité qui, sans lui, devrait être dupliquée dans chaque contrôleur (bloc try/catch autour de chaque appel de service) — c'est ce mécanisme qui permet, par exemple, au frontend d'afficher directement le message « Un événement existe déjà le [date] à [lieu]. » renvoyé par EventService lorsqu'un doublon RG-08 est détecté, plutôt qu'un message d'erreur générique.

### 3.3.2. Attribution automatique des photos (EventImageService)

L'administrateur de Terra Sana ne dispose généralement pas d'une URL d'image sous la main au moment de créer un événement ou un projet. Plutôt que d'afficher une image par défaut identique pour toutes les activités, EventImageService (et son équivalent ProjectImageService) recherche des mots-clés dans le titre et la description de l'événement, et associe la première catégorie thématique correspondante à une photo Unsplash déjà vérifiée — une quarantaine de catégories sont couvertes (cuisine, distribution alimentaire, sport, formation, nettoyage, marché bio, etc.).

Deux détails d'implémentation évitent les faux positifs les plus fréquents : le mot-clé « don » est volontairement exclu au profit de « dons »/« don alimentaire », car « don » est un sous-mot de l'adverbe très courant « donc » ; et « distribution » seul est trop générique (distribution de magazines n'a rien à voir avec de la nourriture), d'où la restriction à « distribution alimentaire ». Lorsqu'une catégorie propose plusieurs photos, la variante retenue est choisie de façon stable — pas aléatoire — via un hachage du texte complet, afin qu'un même événement affiche toujours la même image d'un rechargement à l'autre, tout en répartissant les événements d'une même catégorie sur plusieurs visuels différents :

```
private String pickVariant(List<String> options, String rawText) {
    if (options.size() == 1) return options.get(0);
    int index = Math.floorMod(rawText.hashCode(), options.size());
    return options.get(index);
}
```

Ce mécanisme reste une heuristique par mots-clés, et non une reconnaissance d'image : un champ URL manuel permet à l'administrateur de le remplacer pour tout événement au sujet réellement inédit.

### 3.3.3. Calcul du niveau de fidélité (updateLevel)

La méthode updateLevel d'AppUserService recalcule le niveau du bénévole après chaque événement affectant le nombre de ses participations confirmées — aussi bien une nouvelle confirmation qu'une désinscription qui en fait perdre une. Le niveau n'est donc jamais figé une fois atteint : il reflète à tout instant le nombre réel de participations confirmées (RG-19).

```
public void updateLevel(AppUser user, long confirmedCount) {
    Level oldLevel = user.getLevel();
```

```

Level newLevel = confirmedCount >= 7 ? Level.OR
                : confirmedCount >= 3 ? Level.ARGENT
                : Level.BRONZE;
user.setLevel(newLevel);
userRepo.save(user);
// L'email n'est envoyé qu'en cas de progression réelle, jamais sur une baisse
if (newLevel.ordinal() > oldLevel.ordinal()) {
    emailService.sendLevelChange(user, newLevel.name());
}
}

```

Le détail qui mérite d'être souligné porte sur l'envoi de l'e-mail de félicitations (RG-20) : la comparaison `newLevel.ordinal() > oldLevel.ordinal()` garantit que ce message n'est déclenché qu'en cas de progression effective. Sans cette vérification, un bénévole dont le niveau redescendrait d'ARGENT à BRONZE après une désinscription recevrait, par erreur, le même e-mail de félicitations qu'une réelle promotion — un cas limite qui n'apparaît pas dans une lecture rapide de la règle de gestion, mais que le code met en évidence.

### 3.3.4. Génération des attestations PDF (PdfService)

La génération des attestations et des exports de liste d'inscrits repose sur iText7, utilisé en mode « bas niveau » (construction directe de paragraphes et de tableaux) plutôt que via un système de templates, pour un contenu qui reste volontairement simple. Le contrôle d'accès à l'attestation, déjà appliqué côté contrôleur, est revérifié une seconde fois à l'intérieur même du service, avant toute génération du document :

```

// Génère l'attestation uniquement si la participation est confirmée et l'événement terminé
public byte[] generateAttestation(AppUser user, Event event, Registration registration) {
    if (registration.getStatus() != RegistrationStatus.CONFIRMED) {
        throw new RuntimeException("Attestation disponible uniquement pour une " +
            "participation confirmée.");
    }
    if (event.getStatus() != EventStatus.FINISHED) {
        throw new RuntimeException("Attestation disponible uniquement après " +
            "la fin de l'événement.");
    }
    // ... construction du document PDF (iText7)
}

```

Cette double vérification (RG-21) illustre un principe déjà rencontré en section 3.5.3 pour le mot de passe : toute règle de gestion touchant à une action sensible est revérifiée au plus près de l'exécution réelle, plutôt que déléguée à un unique point de contrôle en amont. La charte graphique du site est reprise jusque dans le document PDF lui-même — l'attestation utilise la même couleur verte (RGB 45, 106, 79) que l'interface, via la classe `DeviceRgb` d'iText7.

### 3.3.5. Envoi asynchrone des e-mails (EmailService)

Le module envoie automatiquement un e-mail à chaque étape clé du parcours bénévole : confirmation d'inscription, refus, promotion depuis la liste d'attente, changement de niveau, annulation d'un événement, message groupé de l'administrateur, réinitialisation de mot de passe. Toutes ces méthodes sont annotées `@Async` : l'aller-retour SMTP réel peut prendre une à trois secondes, un délai qui serait perceptible et bloquant s'il fallait l'attendre avant de répondre à la requête HTTP qui l'a déclenché. Grâce à `@Async`, le contrôleur retourne sa réponse (par exemple la confirmation d'une inscription) immédiatement, pendant que l'envoi de l'e-mail se poursuit sur un thread séparé, géré par le pool de tâches asynchrones de Spring.

### 3.3.6. Interface multilingue (FR / EN / NL)

Chaque page du frontend, y compris l'ensemble du tableau de bord administrateur, définit un objet de traduction `T = { fr: {...}, en: {...}, nl: {...} }`, et récupère les textes affichés via `t = T[lang] || T.fr`, où `lang` est transmis depuis le sélecteur de langue de la barre de navigation jusqu'à chaque route dans `App.js`. Les libellés

variables (par exemple « 3 places restantes sur 5 ») passent par des fonctions plutôt que par du texte figé, afin de rester grammaticalement correctes dans les trois langues :

```
placesRemaining: (n, max) => `${n} places restantes sur ${max}`,  
// ...  
remaining: (n, max) => `${n} of ${max} spots left`,
```

Les données propres à Terra Sana (titres d'événements, descriptions saisies par l'administrateur) ne sont volontairement pas traduites automatiquement : seule l'interface elle-même — labels, boutons, messages de succès ou d'erreur — l'est, ce qui évite d'introduire une dépendance à un service de traduction externe pour du contenu géré par l'association elle-même.

### 3.3.7. Pagination et filtres avancés

Les listes d'événements et de bénévoles côté administrateur reposent sur l'interface Pageable de Spring Data JPA plutôt que sur un chargement complet suivi d'une pagination côté client : chaque appel à `/api/events/paged` ou `/api/volunteers/paged` précise un numéro de page et une taille, et seule la page demandée est effectivement extraite de la base de données. Ce choix évite de transférer l'intégralité des bénévoles ou des événements passés à chaque ouverture du tableau de bord, un point qui deviendra sensible si Terra Sana accumule plusieurs années d'historique d'événements.

Le filtrage des bénévoles par compétence, disponibilité et langue (section 3.4.6, onglet Bénévoles) combine plusieurs conditions optionnelles dans une même requête JPA : lorsque l'administrateur laisse un champ de filtre vide, la condition correspondante est simplement omise plutôt que comparée à une chaîne vide, afin de retourner l'ensemble des bénévoles quand aucun critère n'est renseigné.

### 3.3.8. Graphiques du tableau de bord (Chart.js)

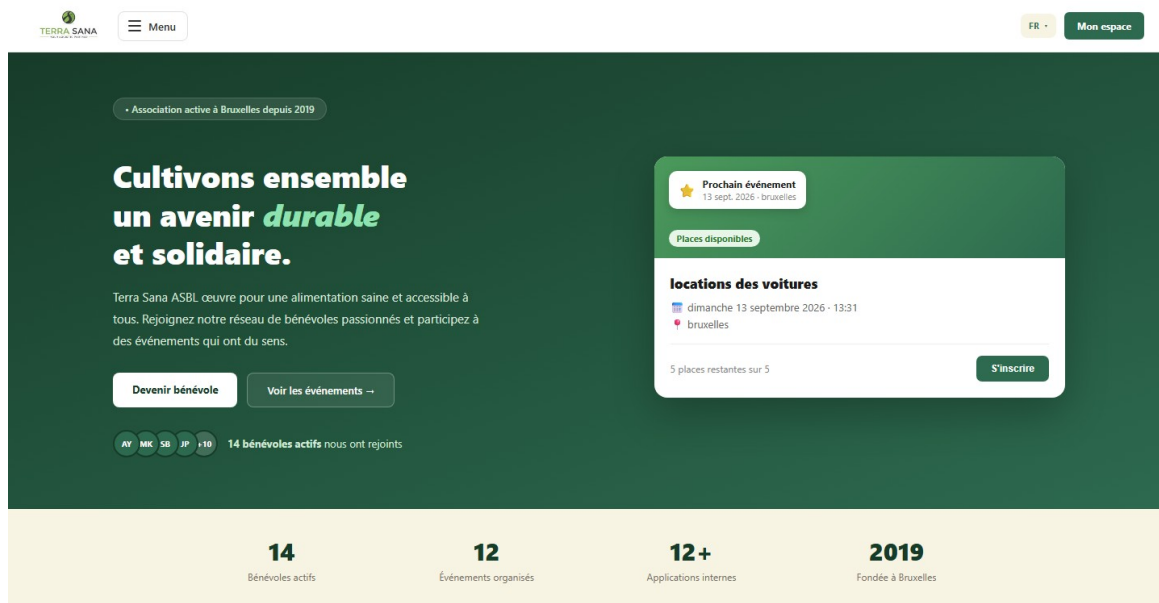
L'onglet Statistiques du tableau de bord administrateur affiche deux visualisations construites avec Chart.js à partir des données réelles de l'association : une courbe (Line) du nombre d'inscriptions par mois sur les six derniers mois, et un diagramme en anneau (Doughnut) répartissant les inscriptions entre CONFIRMED, WAITING et REFUSED, permettant de visualiser en un coup d'œil le taux de participation global. Ces deux graphiques sont recalculés côté frontend à chaque chargement, à partir des inscriptions récupérées via l'API — aucune agrégation statistique n'est pré-calculée ni stockée côté serveur, ce qui garantit que les chiffres affichés reflètent toujours l'état courant de la base.

## 3.4. Présentation des interfaces

Cette section présente les principaux écrans du module, tels qu'implémentés, avec une capture d'écran et une explication fonctionnelle pour chacun.

### 3.4.1. Page d'accueil

La page d'accueil met en avant le prochain événement à venir, les statistiques de l'association (bénévoles actifs, événements organisés, applications internes), la mission de Terra Sana, un parcours en trois étapes pour devenir bénévole, les prochains événements et le hub des douze applications internes héritées du stage.



### 3.4.2. Inscription et connexion bénévole

Un visiteur crée un compte bénévole via un formulaire dédié : nom, prénom, email et mot de passe sont obligatoires (marqués d'un badge « ? »), les autres champs (téléphone, ville, compétences, disponibilités, langue préférée) restent facultatifs, conformément au principe de minimisation des données. Une fois le compte créé, le bénévole se connecte via un jeton JWT distinct de celui de l'administrateur (RG-25).

TERRA SANA

Menu

FR · Espace administrateur

Se connecter

Je m'inscris

TerraSana

Créer un compte bénévole

Rejoignez la communauté Terra Sana

Prénom

Prénom

Nom

Nom

Email

votre@email.com

Mot de passe (min. 8 caractères)

Choisissez un mot de passe

Téléphone

+32 xxx xx xx xx

Date de naissance

dd-mm-yyyy

Genre

— Sélectionner —

Ville

Bruxelles

Code postal

1000

Compétences

Ex : cuisine, communication, logistique...

Disponibilités

Ex : week-ends, mercredis après-midi...

Langue préférée

Français

Créer mon compte

Déjà un compte ? [Se connecter](#)

Terra Sana ASBL

Association bruxelloise pour une alimentation locale, saine et accessible à tous depuis 2019.

ASSOCIATION

À propos de Terra Sana

Notre mission

Nos partenaires

Nous contacter

BÉNÉVOLES

Devenir bénévole

Calendrier des événements

Mon espace personnel

FAQ

CONTACT

S/O, 1200 Woluwe Saint-Lambert

Bruxelles, Belgique

Terrasana@outlook.be

Lun - Ven : 8h00 - 16h00

© 2023 Terra Sana ASBL · Tous droits réservés

Confidentialité · Conditions · Politique cookies

TERRA SANA

Menu

FR · Espace administrateur

Se connecter

Je m'inscris

TerraSana

Espace Bénévole

Connectez-vous pour accéder à vos événements

Email

votre@email.com

Mot de passe

Votre mot de passe

Se connecter

[Mot de passe oublié ?](#)

Pas encore de compte ? [Créer un compte](#)

Terra Sana ASBL

Association bruxelloise pour une alimentation locale, saine et accessible à tous depuis 2019.

ASSOCIATION

À propos de Terra Sana

Notre mission

Nos partenaires

Nous contacter

BÉNÉVOLES

Devenir bénévole

Calendrier des événements

Mon espace personnel

FAQ

CONTACT

S/O, 1200 Woluwe Saint-Lambert

Bruxelles, Belgique

Terrasana@outlook.be

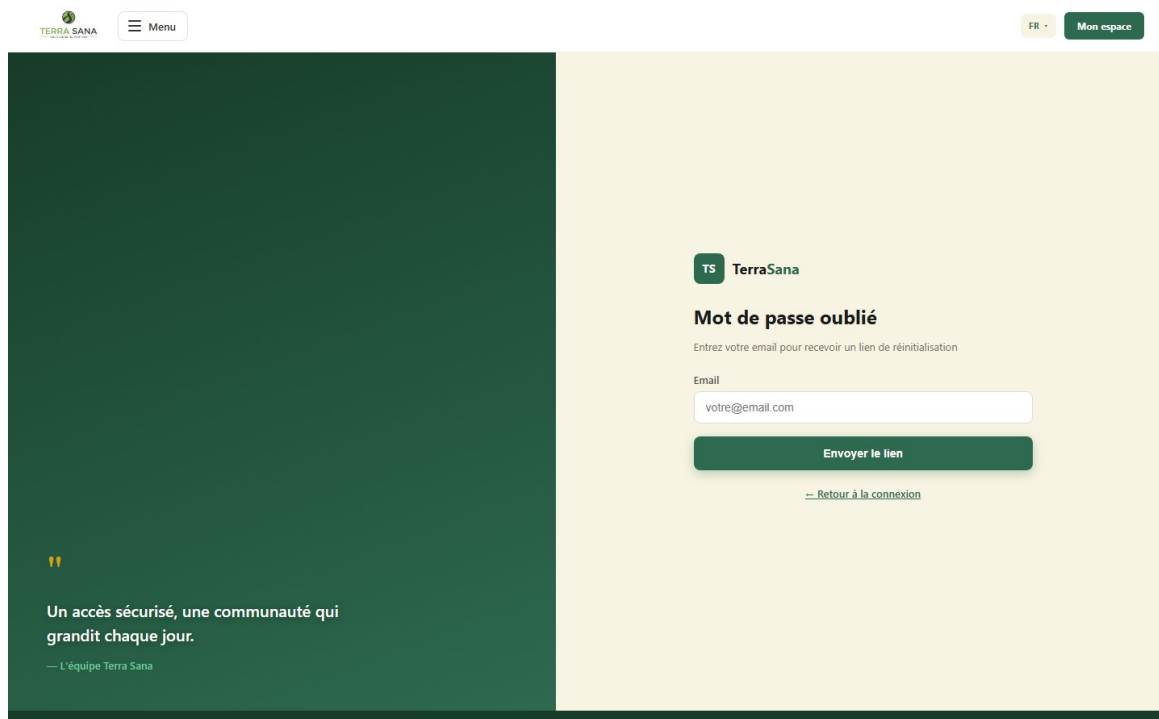
Lun - Ven : 8h00 - 16h00

© 2023 Terra Sana ASBL · Tous droits réservés

Confidentialité · Conditions · Politique cookies

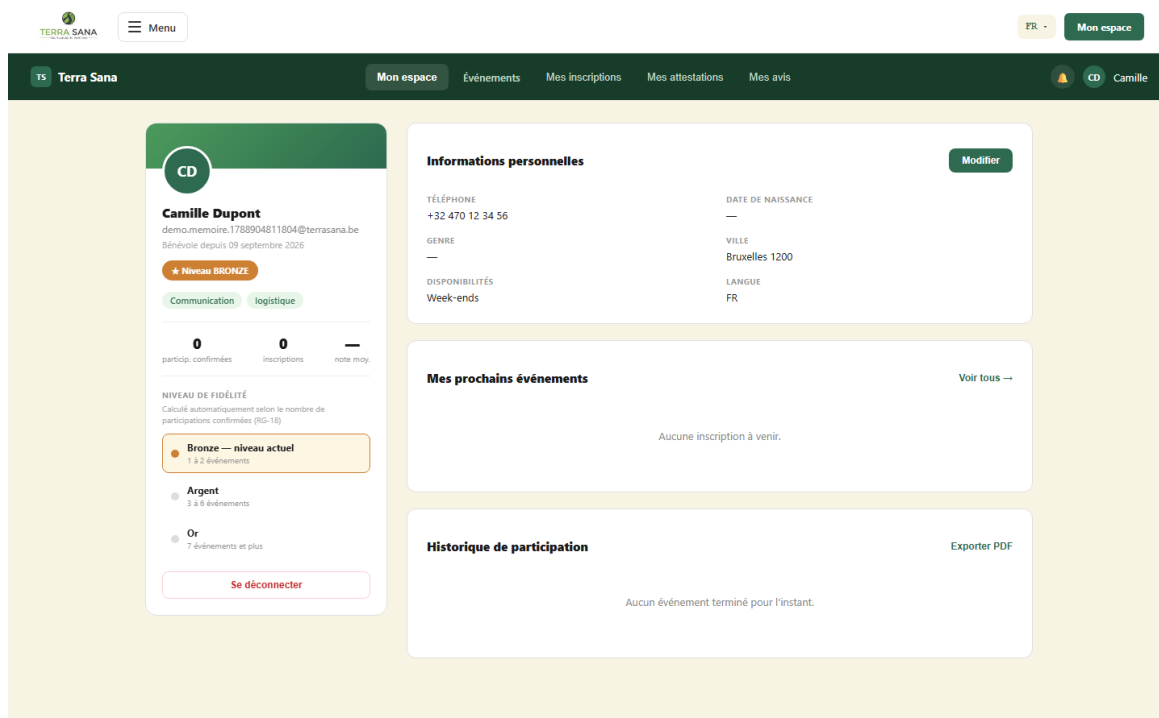
Alain Youndjeu Tchouapi — Épreuve Intégrée

32



### 3.4.3. Espace personnel bénévole

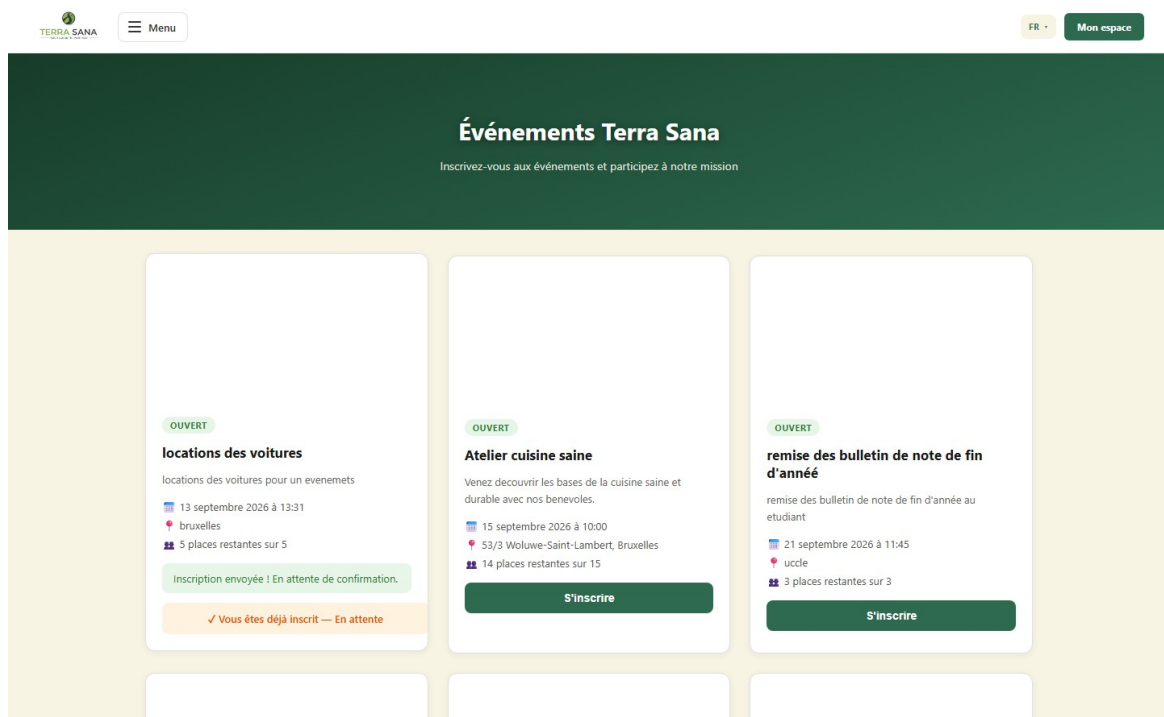
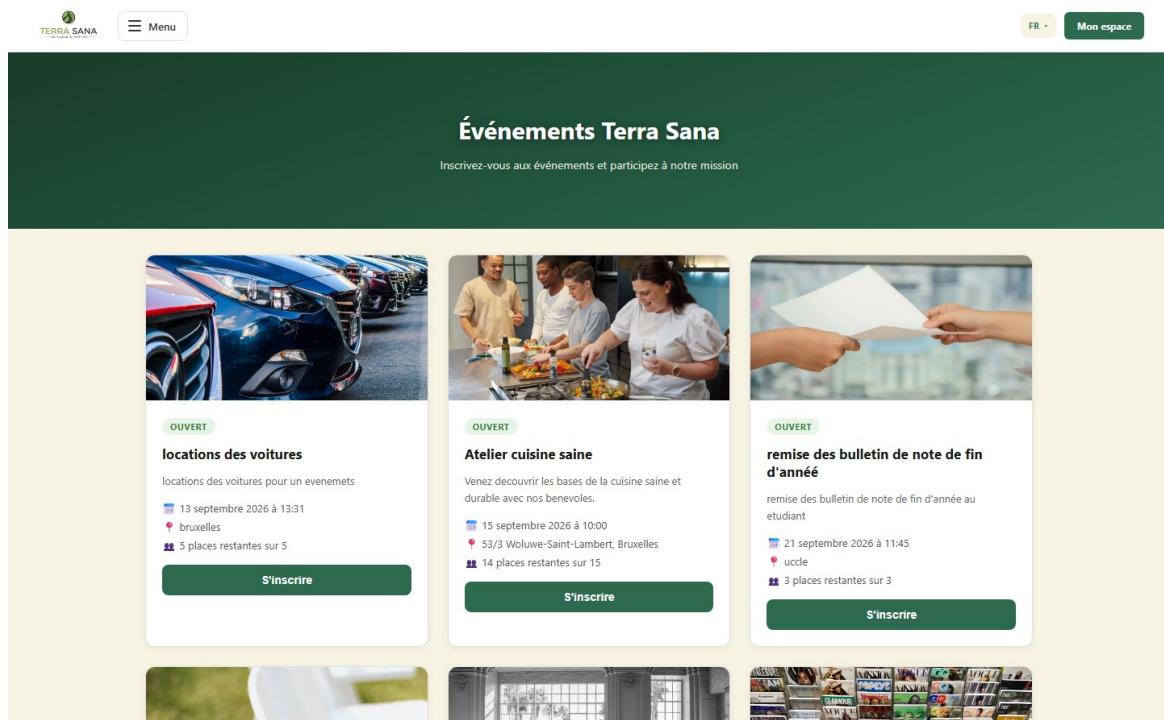
Après connexion, le bénévole accède à son espace personnel : profil avec badge de niveau (Bronze / Argent / Or, RG-19), compétences déclarées, statistiques de participation, progression vers le niveau suivant, informations personnelles modifiables, prochains événements auxquels il est inscrit et historique de participation exportable en PDF.



### 3.4.4. Liste des événements

La page publique des événements affiche l'ensemble des activités à venir sous forme de cartes (photo, statut, date, lieu, places restantes) ainsi qu'une section « Événements passés » distincte, avec le statut TERMINÉ ou

ANNULÉ. Chaque carte permet de s'inscrire directement en un clic. Une fois l'inscription envoyée, la carte affiche immédiatement un retour visuel — confirmation d'envoi, puis rappel que le bénévole est déjà inscrit et en attente de validation par l'administrateur.



### 3.4.5. Parcours complet d'un bénévole

Pour donner une vision d'ensemble cohérente des écrans présentés séparément ci-dessus, cette section retrace le parcours complet d'un bénévole, de sa première visite jusqu'à l'obtention de son attestation, en reliant chaque étape à l'écran et à la règle de gestion qui la régissent.

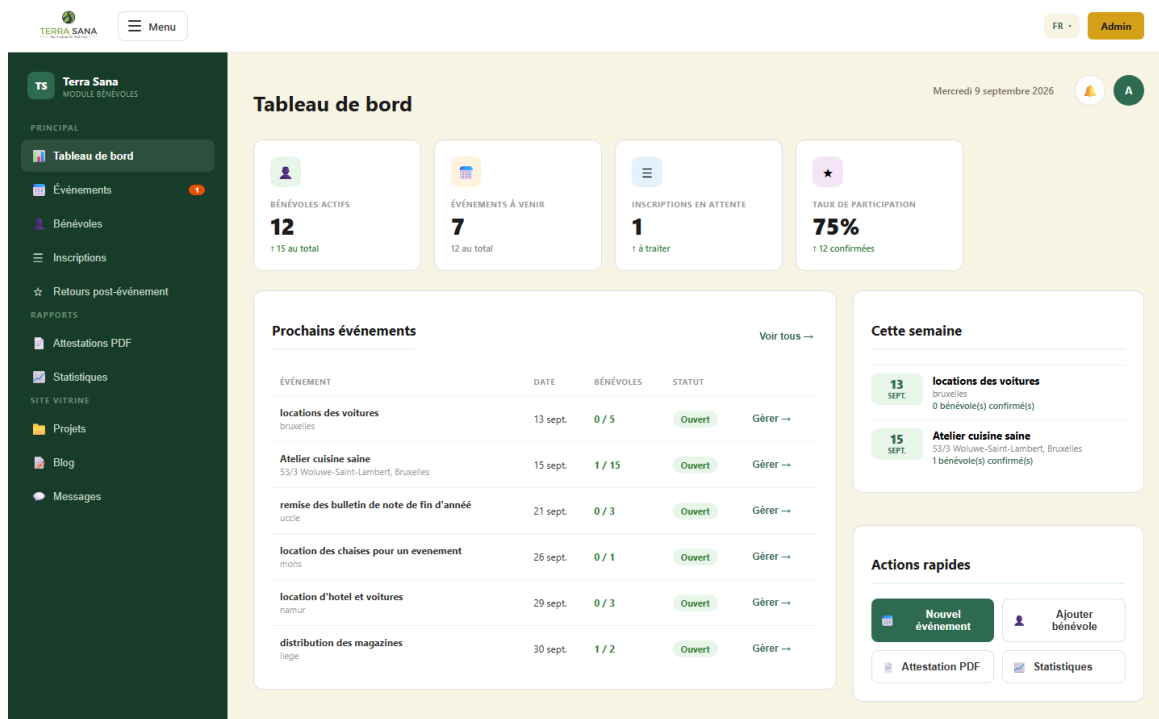
| Étape | Écran / action                                                                         | Règle(s) de gestion associées |
|-------|----------------------------------------------------------------------------------------|-------------------------------|
| 1     | Le visiteur consulte la liste des événements sans être connecté                        | Consultation publique         |
| 2     | Il crée un compte via le formulaire d'inscription (nom, email, mot de passe)           | RG-01, RG-02                  |
| 3     | Il se connecte et arrive sur son espace personnel, niveau BRONZE par défaut            | RG-19                         |
| 4     | Il s'inscrit à un événement ; si celui-ci est complet, il est placé en liste d'attente | RG-09, RG-10                  |
| 5     | Il reçoit un e-mail automatique confirmant son inscription ou sa mise en attente       | RG-12                         |
| 6     | L'administrateur confirme l'inscription depuis le tableau de bord                      | RG-11                         |
| 7     | Le jour de l'événement passé, son statut passe à FINISHED                              | RG-06                         |
| 8     | Le bénévole laisse un avis noté (1 à 5) sur l'événement                                | RG-15 à RG-18                 |
| 9     | Sa participation confirmée fait progresser son compteur de niveau                      | RG-19, RG-20                  |
| 10    | Il télécharge son attestation de participation au format PDF                           | RG-21                         |

*Tableau 11 — Parcours complet d'un bénévole, du premier contact à l'attestation*

### 3.4.6. Tableau de bord administrateur

Le tableau de bord administrateur, accessible uniquement au rôle ADMIN (RG-24), centralise la gestion opérationnelle du module. Onze onglets organisent cette gestion, répartis en trois groupes dans la barre latérale : Principal (tableau de bord, événements, bénévoles, inscriptions, retours post-événement), Rapports (attestations PDF, statistiques) et Site vitrine (projets, blog, messages — ces trois derniers hérités du site développé pendant le stage). Les sous-sections suivantes détaillent chacun des écrans propres au module TFE, avec une capture d'écran réelle et son fonctionnement.

La page d'accueil du tableau de bord affiche une vue synthétique de l'activité : quatre indicateurs clés (bénévoles actifs, événements à venir, inscriptions en attente, taux de participation), la liste des prochains événements avec leur taux de remplissage, et des raccourcis vers les actions les plus fréquentes.



L'onglet Événements réunit, sur un même écran, le formulaire de création (titre, lieu, description, date, capacité maximale) et la liste des événements existants, avec pour chacun un sélecteur de statut manuel (utile pour une annulation, RG-06) et des actions rapides : voir les inscrits, voir les avis, envoyer un email groupé, modifier ou supprimer. La capture ci-dessous montre le formulaire rempli pour un événement de démonstration, juste avant sa création.

**Événements**

Mercredi 9 septembre 2026

**Créer un événement**

Atelier compost partagé Parc de Woluwe, Bruxelles

Venez apprendre à composter vos déchets organiques avec les bénévoles de Terra Sana.

Date et heure: 14-11-2026 10:00

Nombre de places max: 20

URL image (optionnel)

**Créer l'événement**

**Liste des événements (12/12)**

Rechercher par titre...

**locations des voitures**

13 sept. 2026, 13:31 | bruxelles | 5 places

Statut: OPEN

Actions: Inscrits, Avis, Email groupé, Modifier, Supprimer

L'onglet Inscriptions centralise, tous événements confondus, la validation manuelle exigée par RG-11 : chaque ligne affiche le bénévole, l'événement concerné et son statut, avec deux boutons Confirmer / Refuser directement disponibles pour toute inscription encore WAITING.

**Inscriptions**

Mercredi 9 septembre 2026

**Suivi des inscriptions**  
1 en attente - 12 confirmées - 3 refusées

**Toutes les inscriptions (16)**

| BÉNÉVOLE                                                 | ÉVÉNEMENT                        | STATUT                                                    |
|----------------------------------------------------------|----------------------------------|-----------------------------------------------------------|
| Julie Martin<br>demo.memoire2.1788908758851@terrasana.be | locations des voitures           | WAITING <a href="#">Confirmer</a> <a href="#">Refuser</a> |
| mai septembre<br>alainyoungjeu@gmail.com                 | distribution des magazines       | CONFIRMED                                                 |
| Test2 Alain<br>alainyoungjeu+test2@gmail.com             | vente des poissons               | CONFIRMED                                                 |
| mai septembre<br>alainyoungjeu@gmail.com                 | marché de jette                  | CONFIRMED                                                 |
| Test2 Alain<br>alainyoungjeu+test2@gmail.com             | marché de jette                  | CONFIRMED                                                 |
| Test2 Alain<br>alainyoungjeu+test2@gmail.com             | pharmacie de garde               | CONFIRMED                                                 |
| Test2 Alain<br>alainyoungjeu+test2@gmail.com             | vente des poissons               | REFUSED                                                   |
| mai septembre<br>alainyoungjeu@gmail.com                 | vente des poissons               | CONFIRMED                                                 |
| mai septembre<br>alainyoungjeu@gmail.com                 | Marché bio et producteurs locaux | REFUSED                                                   |

L'onglet Bénévoles liste l'ensemble des comptes inscrits, avec leur niveau de fidélité (RG-19), leur statut actif/inactif et un bouton de désactivation conforme à RG-03 (le compte est neutralisé, jamais supprimé, pour préserver son historique). Trois filtres — compétence, disponibilité, langue — permettent à l'administrateur de cibler rapidement un profil recherché pour un événement particulier.

**Bénévoles**

Mercredi 9 septembre 2026

**Bénévoles inscrits (15/15)**

Filtrer par compétence... Filtrer par disponibilité... Toutes les langues

**Alain Youndjeu** **BRONZE** **Actif** Inscrit le 21 août 2026, 00:43 [Désactiver](#)

test@terrasana.be  
+32 470 00 00 00

Compétences : Communication

Disponibilités : Week-ends

**Sophie Lambert** **BRONZE** **Inactif** Inscrit le 21 août 2026, 03:45 [Réactiver](#)

sophie.lambert@demo.be

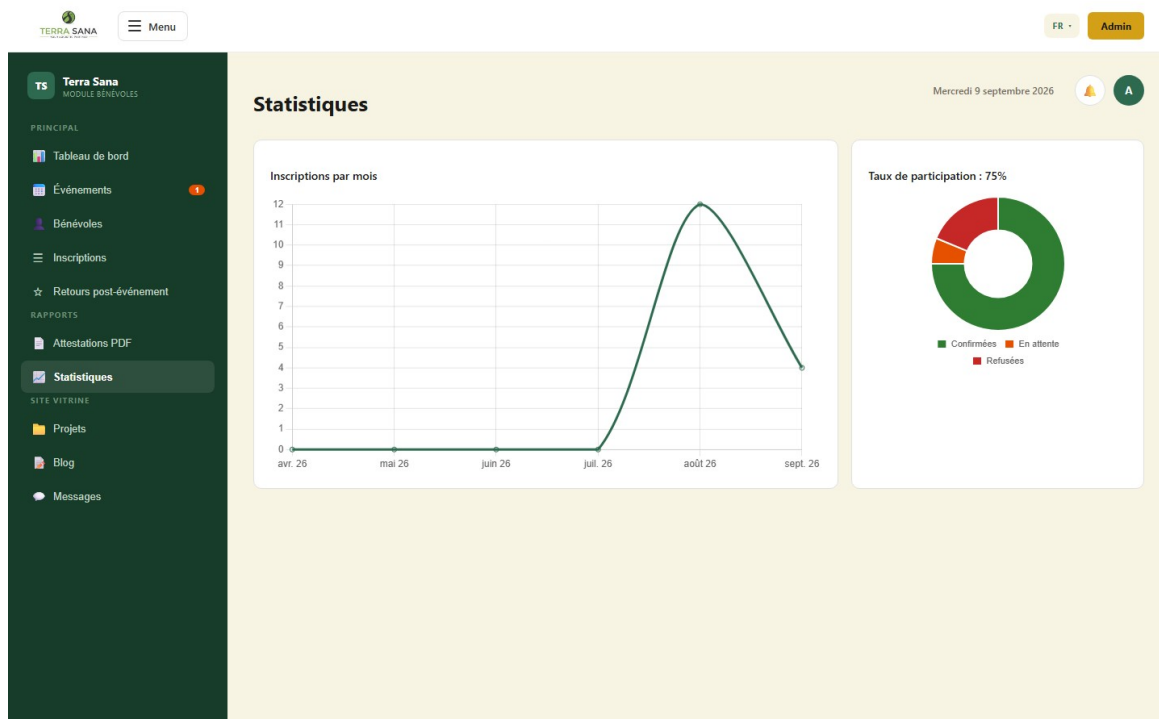
Compétences : Cuisine, animation

Disponibilités : Week-ends

**Marc Dubois** **BRONZE** **Inactif** Inscrit le 21 août 2026, 03:45 [Réactiver](#)

marc.dubois@demo.be

L'onglet Statistiques restitue graphiquement, via Chart.js (section 3.3.8), l'évolution des inscriptions sur les six derniers mois et la répartition des statuts d'inscription — une lecture immédiate de l'activité de l'association qu'aucun fichier Excel ne permettait auparavant (section 1.3).



Enfin, l'onglet Attestations PDF liste les bénévoles éligibles à une attestation de participation — c'est-à-dire ceux dont l'inscription est CONFIRMED à un événement déjà FINISHED (RG-21) — à titre indicatif pour l'administrateur : chaque bénévole télécharge lui-même son propre document depuis son espace personnel (section 3.4.3).

**Attestations PDF**

Mercredi 9 septembre 2026

**Bénévoles éligibles à une attestation**

Participation confirmée à un événement terminé (RG-20) — le bénévole télécharge lui-même son attestation depuis son espace personnel.

| BÉNÉVOLE      | ÉVÉNEMENT          | DATE       |
|---------------|--------------------|------------|
| juil mai      | marché du midi     | 30/08/2026 |
| mai septembre | marché du midi     | 30/08/2026 |
| mai septembre | vente des poissons | 27/08/2026 |
| Test2 Alain   | marché de jette    | 02/09/2026 |
| mai septembre | marché de jette    | 02/09/2026 |
| Test2 Alain   | vente des poissons | 03/09/2026 |

### 3.4.7. Pages institutionnelles et interface multilingue

Les pages du site vitrine hérité du stage (À propos, Contact, Aide/FAQ...) ont été redessinées avec la nouvelle charte graphique et restent, elles aussi, entièrement traduites en trois langues — y compris les questions fréquentes, comme l'illustre la capture ci-dessous, prise après bascule du sélecteur de langue sur l'anglais.

## A propos de Terra Sana

### Notre mission

Terra Sana ASBL est une organisation à but non lucratif fondée en 2019 à Bruxelles. Elle soutient les producteurs locaux et promote le circuit court.

### Nos valeurs

- Soutien aux producteurs locaux
- Promotion du circuit court
- Réduction des déchets et recyclage
- Accompagnement des personnes en décrochage
- Encouragement des pratiques durables

### Notre équipe

Notre équipe est composée d'ouvriers en réinsertion, d'employés, de bénévoles et de stagiaires.

## Nous contacter

Envoyez-nous un message, nous répondrons rapidement

Nom complet

Email

Message

0 / 1000

Votre message (minimum 20 caractères)...

Envoyer le message

## Help centre

Find answers to the most frequently asked questions

### How do I access the applications?

Go to the Projects page and click the Open button of the application you want.

### How do I send a message to Terra Sana?

Go to the Contact page, fill in the form and click Send message.

### How do I apply as a volunteer or intern?

Go to the Volunteering and Internship page and fill in the application form.

### How do I log in to the admin area?

Click Log in in the navigation bar and enter your administrator credentials.

### What languages is the site available in?

The site is available in French, English and Dutch. Use the language selector in the navigation bar.

### Couldn't find your answer?

Contact us directly at [Terrasana@outlook.be](mailto:Terrasana@outlook.be)

Les pages Sponsors et Bénévolat complètent la vitrine institutionnelle : la première présente les partenaires qui soutiennent Terra Sana (Bruxelles Environnement, la Région de Bruxelles, la commune de Woluwe-Saint-Lambert), la seconde constitue le point d'entrée principal vers la création d'un compte bénévole pour un nouveau visiteur.

## Nos sponsors

Partenaires qui soutiennent la mission de Terra Sana

**B**

PARTENAIRE OFFICIEL

**Bruxelles Environnement**

Agence bruxelloise pour  
l'environnement et l'énergie

**R**

SOUTIEN INSTITUTIONNEL

**Région de Bruxelles**

Soutien institutionnel de la Région de  
Bruxelles-Capitale

**C**

PARTENAIRE LOCAL

**Commune de Woluwe**

Partenariat avec la commune de  
Woluwe-Saint-Lambert

## Devenir sponsor

Vous souhaitez soutenir Terra Sana ? Contactez-nous !

[Nous contacter](#)

## 3.5. Sécurité

La sécurité du module s'appuie sur Spring Security et une authentification JWT entièrement stateless : aucune session n'est conservée côté serveur, chaque requête porte sa propre preuve d'identité dans l'en-tête HTTP Authorization.

### 3.5.1. Deux jetons distincts (RG-25)

Conformément à RG-25, les bénévoles et l'administrateur reçoivent chacun un jeton signé avec un secret distinct (JwtUtil pour l'admin, VolunteerJwtUtil pour le bénévole), tous deux valables 24 heures (RG-23). Le filtre JwtFilter tente successivement les deux JwtUtil lors de la validation d'une requête et assigne le rôle ROLE\_ADMIN ou ROLE\_VOLUNTEER en conséquence — le rôle n'est pas un claim porté par le jeton lui-même, il est déduit de la clé de signature qui a permis de le valider. Ce filtre, exécuté une seule fois par requête (OncePerRequestFilter), s'intercale avant le filtre d'authentification standard de Spring Security :

```
protected void doFilterInternal(HttpServletRequest request,
    HttpServletResponse response, FilterChain chain) {
    String header = request.getHeader("Authorization");
    if (header != null && header.startsWith("Bearer ")) {
        String token = header.substring(7);
        // Admin et bénévole utilisent des clés secrètes distinctes : on essaie les deux
        if (jwtUtil.validateToken(token)) {
            authenticate(jwtUtil.extractUsername(token), "ROLE_ADMIN");
        } else if (volunteerJwtUtil.validateToken(token)) {
            authenticate(volunteerJwtUtil.extractUsername(token), "ROLE_VOLUNTEER");
        }
    }
    chain.doFilter(request, response);
}
```

### 3.5.2. Autorisation par endpoint

La configuration Spring Security (SecurityConfig) définit explicitement, endpoint par endpoint et par méthode HTTP, les routes publiques, celles réservées au rôle ADMIN, et celles ouvertes à tout jeton valide

(chaque contrôleur vérifiant alors lui-même l'identité exacte, par exemple pour n'autoriser un bénévole à annuler que sa propre inscription).

| Portée                           | Endpoint                                                    | Règle                                       |
|----------------------------------|-------------------------------------------------------------|---------------------------------------------|
| Public                           | POST /api/volunteers/register                               | Inscription (RG-01)                         |
| Public                           | POST /api/volunteers/login, forgot-password, reset-password | Authentification bénévole                   |
| Public                           | GET /api/events, /api/events/*                              | Consultation des événements (visiteur)      |
| Public                           | GET /api/projects, /api/posts                               | Site vitrine hérité du stage                |
| Public                           | GET /api/reviews/event/*                                    | Moyenne des avis affichée publiquement      |
| Public                           | POST /api/contact                                           | Formulaire de contact                       |
| Jeton bénévole                   | GET/PUT /api/volunteers/me                                  | Consultation et modification du profil      |
| Jeton bénévole                   | GET /api/volunteers/me/attestation/*                        | Téléchargement d'attestation (RG-21)        |
| Réservé ADMIN                    | POST/PUT/DELETE /api/events/**                              | Gestion des événements (RG-24)              |
| Réservé ADMIN                    | PUT /api/registrations/*/confirm, /reject                   | Validation des inscriptions (RG-11, RG-24)  |
| Réservé ADMIN                    | GET /api/volunteers, /api/volunteers/paged                  | Liste et filtres des bénévoles              |
| Réservé ADMIN                    | PUT /api/volunteers/*/toggle-active                         | Activation / désactivation (RG-03)          |
| Réservé ADMIN                    | POST /api/registrations/event/*/group-email                 | Envoi d'un email groupé                     |
| Jeton valide (bénévole ou admin) | Le reste des endpoints                                      | Identité exacte vérifiée dans le contrôleur |

Tableau 12 — Règles d'autorisation par endpoint (SecurityConfig)

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers(HttpMethod.POST, "/api/volunteers/register").permitAll()
    .requestMatchers(HttpMethod.GET, "/api/events", "/api/events/*").permitAll()
    .requestMatchers(HttpMethod.POST, "/api/events").hasRole("ADMIN")
    .requestMatchers(HttpMethod.PUT, "/api/registrations/*/confirm").hasRole("ADMIN")
    .requestMatchers("/**").permitAll() // le contrôleur vérifie l'identité exacte
)
.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
```

### 3.5.3. Hachage des mots de passe et validation côté serveur

Les mots de passe ne sont jamais stockés en clair : BCryptPasswordEncoder les hache à l'inscription et vérifie l'authenticité à la connexion, avec un sel intégré automatiquement (RG-02). La longueur minimale de 8 caractères imposée par RG-02 est vérifiée deux fois : une première fois côté frontend (attribut minLength du formulaire, pour un retour immédiat à l'utilisateur), et une seconde fois, indépendamment, côté backend dans AppUserService — car un attribut HTML n'empêche en rien un appel direct à l'API contournant le formulaire :

```
public AppUser register(AppUser user) {
    if (userRepo.existsByEmail(user.getEmail())) {
        throw new RuntimeException("Un compte existe déjà avec cet email.");
    }
    if (user.getPassword() == null || user.getPassword().length() < 8) {
        throw new RuntimeException("Le mot de passe doit contenir au moins 8 caractères.");
    }
    user.setPassword(passwordEncoder.encode(user.getPassword()));
    user.setLevel(Level.BRONZE);
}
```

```
return userRepo.save(user);
}
```

Ce principe — ne jamais faire confiance à la seule validation du navigateur — est appliqué systématiquement dans les services du module, chaque règle de gestion touchant à une donnée saisie par l'utilisateur étant revérifiée côté serveur avant toute écriture en base.

### 3.5.4. Configuration CORS

La configuration CORS est appliquée globalement au niveau de Spring Security lui-même — et pas seulement via l'annotation `@CrossOrigin` — afin que les réponses 401/403 générées par Security elle-même portent également les en-têtes CORS attendus par le navigateur.

## 3.6. Environnement de développement et de démonstration

Conformément au plan de travail validé avec Madame Namur, l'application est développée et présentée en environnement local, via WAMP Server sous Windows (Apache, MySQL, PHP côté outillage local) : aucun déploiement en ligne (type Railway ou Vercel) n'est prévu dans le cadre de ce TFE. Ce choix permet de concentrer l'effort sur la qualité de l'analyse, la conformité aux bonnes pratiques et la robustesse fonctionnelle du module, plutôt que sur la configuration d'une infrastructure cloud.

Le code source est versionné sur GitHub tout au long du développement (dépôt indiqué en Annexe C), avec des commits réguliers documentant chaque étape plutôt qu'un unique commit final. Cette discipline de commit fréquent s'est révélée particulièrement utile après un incident survenu en cours de projet : un outil de migration automatisée a un jour remisé (git stash) l'ensemble des modifications non committées du backend sans prévenir, ce qui aurait pu faire disparaître plusieurs jours de travail si les changements avaient été accumulés dans un unique commit prévu en fin de journée plutôt que validés au fil de l'eau. L'incident, documenté plus en détail en section 4.2, a renforcé la rigueur apportée au versionnement pour le reste du développement.

### 3.6.1. Mise en route locale

Reproduire l'environnement de démonstration ne nécessite que WAMP Server, Node.js et un IDE Java. Les étapes suivantes permettent de lancer le module depuis une copie du dépôt GitHub (Annexe C) :

| Étape                    | Commande / action                                                                                                                     |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| 1. Base de données       | Démarrer WAMP Server, créer la base terresana via phpMyAdmin, exécuter le script de l'Annexe A                                        |
| 2. Configuration backend | Renseigner <code>application.properties</code> : URL JDBC, identifiants MySQL, secrets JWT, identifiants Gmail SMTP                   |
| 3. Démarrage backend     | <code>mvn spring-boot:run</code> depuis le dossier backend/ — API disponible sur <code>http://localhost:8080</code>                   |
| 4. Démarrage frontend    | <code>npm install</code> puis <code>npm start</code> depuis le dossier frontend/ — application sur <code>http://localhost:3000</code> |
| 5. Compte administrateur | Un compte admin est inséré directement en base (table admin) pour la première connexion                                               |

Tableau 13 — Étapes de mise en route de l'environnement local

### 3.6.2. Vérification et tests manuels

En l'absence de suite de tests automatisés (limite reconnue en section 4.3), la conformité de chaque règle de gestion a été vérifiée manuellement, directement dans le navigateur, à chaque étape significative du développement : création d'un événement à une date et un lieu déjà utilisés pour confirmer le rejet attendu (RG-08), inscription à un événement complet pour vérifier le placement en liste d'attente (RG-10),

désinscription d'un bénévole confirmé pour observer la promotion automatique du suivant (RG-13), et bascule systématique entre les trois langues de l'interface pour s'assurer qu'aucun texte ne restait figé en français après un changement de langue.

Cette démarche, bien que manuelle, a permis d'identifier et de corriger plusieurs anomalies avant la rédaction de ce rapport — notamment un premier jet de la règle RG-08 qui ne comparait que le titre et l'horaire exacts des événements, insuffisant pour détecter un doublon portant un intitulé différent au même lieu et à la même date, corrigé après qu'un contre-exemple concret a mis le problème en évidence.

Le tableau ci-dessous reprend un échantillon représentatif des scénarios rejoués manuellement, sélectionnés pour couvrir un cas nominal et un cas limite pour chacun des groupes de règles de gestion les plus sensibles.

| Scénario testé                                                                                               | Résultat attendu                                                     | Résultat constaté |
|--------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|-------------------|
| Créer un événement à une date et un lieu déjà utilisés par un autre événement, avec un titre différent       | Rejet avec message précisant la date et le lieu en conflit (RG-08)   | Conforme          |
| S'inscrire deux fois de suite au même événement avec le même compte                                          | Rejet : contrainte d'unicité (RG-09)                                 | Conforme          |
| S'inscrire à un événement dont toutes les places CONFIRMED sont prises                                       | Placement automatique en liste d'attente, position calculée (RG-10)  | Conforme          |
| Confirmer une inscription en attente alors que l'événement vient d'atteindre sa capacité maximale entretemps | Rejet explicite, aucun dépassement de capacité (section 2.3)         | Conforme          |
| Désinscrire un bénévole CONFIRMED alors qu'un autre bénévole est en liste d'attente                          | Promotion automatique du premier en attente, email envoyé (RG-13)    | Conforme          |
| Laisser un avis sur un événement dont le statut n'est pas encore FINISHED                                    | Rejet (RG-16)                                                        | Conforme          |
| Laisser un second avis sur un événement déjà noté par le même bénévole                                       | Rejet : contrainte d'unicité (RG-17)                                 | Conforme          |
| Appeler un endpoint réservé à l'administrateur avec un jeton bénévole valide                                 | Refus d'accès (RG-24)                                                | Conforme          |
| Changer la langue de l'interface pendant la consultation du tableau de bord administrateur                   | Bascule immédiate de l'ensemble des libellés, sans perte de contexte | Conforme          |

Tableau 14 — Échantillon des scénarios de vérification manuelle rejoués avant remise du rapport

## IV. Conclusion

### 4.1. Apports personnels

Le travail réalisé durant le stage (site vitrine, hub des applications, authentification administrateur, base de données de départ) constitue la fondation technique du projet. Toutes les fonctionnalités présentées dans ce rapport sont, elles, entièrement conçues et développées par mes soins, à domicile, après le stage, à partir de cette base. Le tableau ci-dessous en donne une vue synthétique et chiffrée.

| Critère                       | Stage                          | TFE (apport personnel)                                                        |
|-------------------------------|--------------------------------|-------------------------------------------------------------------------------|
| Tables en base de données     | 4                              | +4 nouvelles (8 au total)                                                     |
| Endpoints API REST            | 17                             | +30 nouveaux (47 au total)                                                    |
| Module bénévoles              | aucun                          | module complet (inscription, profil, niveaux)                                 |
| Gestion des événements        | aucune                         | module complet (CRUD, inscriptions, retours)                                  |
| Envois d'e-mails automatiques | réponses au contact uniquement | confirmation, liste d'attente, groupe, réinitialisation, changement de niveau |
| Export PDF                    | aucun                          | attestation bénévole + liste des inscrits par événement                       |

Tableau 15 — Comparaison entre le travail de stage et l'apport personnel du TFE

Sur le plan technique, ce travail m'a permis d'approfondir Spring Security dans un contexte à deux rôles (bénévole / administrateur) avec des jetons JWT distincts, de concevoir une base de données relationnelle conforme aux bonnes pratiques de normalisation en partant du diagramme de classes jusqu'au script SQL physique, et d'intégrer des bibliothèques que je n'avais pas utilisées auparavant (iText7 pour la génération de PDF, Chart.js pour les graphiques du tableau de bord).

Sur le plan méthodologique, il m'a surtout appris à faire évoluer une règle de gestion à partir d'un cas concret plutôt que dans l'abstrait : la règle de détection des doublons d'événements (RG-08), par exemple, a été affinée après qu'un contre-exemple précis a montré qu'une simple comparaison de titre et d'horaire était insuffisante — seule la combinaison date + lieu, indépendamment du titre, capture réellement l'intention de la règle. Cette itération m'a appris à ne jamais considérer une règle de gestion comme définitive tant qu'elle n'a pas été confrontée à un cas réel, aussi simple paraisse-t-elle sur papier.

Enfin, la nécessité de traduire l'intégralité de l'interface — y compris un tableau de bord administrateur de plus de 1400 lignes, jamais pensé au départ pour être multilingue — a été une leçon de rigueur en elle-même : elle a montré qu'une fonctionnalité transversale comme l'internationalisation ne peut pas rester une réflexion secondaire ajoutée en fin de projet, sous peine de devoir reprendre l'ensemble des écrans un à un.

Sur le plan humain, ce travail a aussi été une première expérience de traduction d'un besoin flou en spécification précise. Les entretiens avec Monsieur Seraye ne livraient pas des règles de gestion toutes faites : une phrase comme « il faudrait qu'on sache qui a vraiment participé » a dû être reformulée, question après question, jusqu'à devenir RG-14 à RG-17 (qui peut laisser un avis, à quel moment, une seule fois, avec quelle échelle de note). C'est cet exercice de reformulation, plus que l'écriture du code elle-même, qui a demandé le plus de rigueur : une règle mal comprise au départ se répercute silencieusement dans le diagramme de classes, puis dans le schéma SQL, puis dans chaque écran qui en dépend.

## 4.2. Difficultés rencontrées

| Difficulté                                                                                                                                                            | Solution mise en œuvre                                                                                                                                                                                                                       |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Traduction incomplète du site en trois langues : certaines pages recevaient la langue sélectionnée sans jamais l'utiliser dans leur contenu                           | Audit systématique de chaque page pour vérifier que la prop lang était non seulement transmise depuis App.js, mais bien exploitée via un objet de traduction T[lang], avec repli sur le français                                             |
| Récupération de travail après une opération de migration automatisée ayant remisé (git stash) des modifications non committées du backend                             | Inspection de git stash list et git reflog pour confirmer qu'aucune donnée n'était réellement perdue, puis restauration fichier par fichier et adoption d'un rythme de commit plus fréquent                                                  |
| Association automatique d'une photo pertinente à un événement ou un projet à partir de son seul titre                                                                 | Système de correspondance par mots-clés avec plusieurs variantes possibles par catégorie, sélectionnées de façon stable via un hachage du titre (Math.floorMod), plutôt qu'une image fixe par catégorie                                      |
| Cohérence stricte entre le diagramme de classes validé et les entités JPA réellement implémentées                                                                     | Vérification systématique des relations directement dans le code source (annotations @ManyToOne, @JoinColumn) avant toute mise à jour du diagramme ou du dictionnaire de données, plutôt que de faire confiance à la seule mémoire du schéma |
| Adaptation du tableau de bord administrateur aux petits écrans : la barre latérale verticale devenait un bandeau horizontal illisible, aux boutons inutilement étirés | Correction ciblée des règles CSS responsives (largeur des pastilles de navigation calculée sur leur contenu, séparateurs entre groupes) sans toucher à l'agencement du mode desktop                                                          |
| Adresse postale de Terra Sana incohérente entre le pied de page du site et le service de génération d'attestations PDF                                                | Recherche de toutes les occurrences de l'adresse dans le code backend (PdfService) et harmonisation sur l'adresse réelle du siège                                                                                                            |

Tableau 16 — Principales difficultés rencontrées et solutions apportées

## 4.3. Regard critique sur le travail réalisé

Le module remplit ses objectifs principaux : il est fonctionnel, sécurisé, documenté et directement utilisable par Terra Sana ASBL pour remplacer la gestion actuelle par fichiers Excel. Un regard honnête permet toutefois d'en identifier les limites.

- L'absence de suite de tests automatisés (JUnit côté backend, tests de composants côté frontend) reste une lacune dans la démarche qualité — un axe d'amélioration prioritaire si le module devait évoluer davantage. La vérification manuelle décrite en section 3.6.2, bien que systématique, ne protège pas contre une régression future introduite par inadvertance.
- L'attribution automatique des photos d'événements et de projets, basée sur une correspondance de mots-clés, reste une heuristique et non une reconnaissance d'image réelle : elle couvre une quarantaine de catégories mais nécessite un filet de sécurité (URL manuelle) pour les sujets réellement inédits.
- Le dashboard administrateur, bien que fonctionnel et désormais entièrement traduit en trois langues, reste pensé pour un usage interne à l'association plutôt que pour un grand nombre d'administrateurs simultanés.
- Le système de niveaux de fidélité (RG-19) reste volontairement simple : il ne distingue pas le type d'événement, un bénévole présent à sept petites collectes atteignant le niveau OR au même titre qu'un bénévole ayant organisé sept grands événements.
- L'environnement 100 % local, choix assumé et validé avec l'encadrement académique (section 3.6), signifie que Terra Sana ne peut pas encore utiliser le module au quotidien sans qu'un poste dédié reste allumé et configuré — un déploiement en ligne reste nécessaire avant tout usage réel par l'association, au-delà du cadre de ce TFE.

## 4.4. Perspectives

Plusieurs évolutions restent envisageables pour prolonger ce travail, une fois la phase de validation locale achevée :

- l'ajout d'une suite de tests automatisés (JUnit 5 / Mockito côté backend, React Testing Library côté frontend) pour sécuriser les évolutions futures des règles de gestion et remplacer, à terme, la vérification manuelle décrite en section 3.6.2 ;
- la mise en place de notifications en temps réel pour l'administrateur (nouvelles inscriptions, places libérées), par exemple via WebSocket, plutôt que la nécessité actuelle de rafraîchir le tableau de bord ;
- un déploiement en ligne du module (Railway ou équivalent pour le backend, Vercel pour le frontend), une fois la phase de validation locale terminée, pour un usage quotidien par Terra Sana ASBL sans dépendance à un poste WAMP allumé ;
- l'extension du système de niveaux de fidélité avec des badges thématiques (par exemple un badge par type d'événement), au-delà du seul comptage Bronze / Argent / Or, pour mieux refléter la diversité de l'engagement bénévole (section 4.3) ;
- l'ouverture du hub des applications internes (actuellement limité au stage) à une authentification unique partagée avec l'espace bénévole, pour que Terra Sana n'ait à terme qu'un seul système de comptes plutôt que deux univers distincts ;
- l'ajout d'une gestion multi-administrateurs avec des droits différenciés, envisagée dès l'analyse (section 1.5) mais volontairement écartée du périmètre pour tenir les délais de la 2ème session ;
- un rappel automatique envoyé aux bénévoles inscrits la veille d'un événement, en s'appuyant sur le mécanisme de tâche planifiée déjà en place pour le passage en FINISHED (section 3.2.3), afin de réduire le taux d'absentéisme constaté sur certains événements.

Aucune de ces pistes ne remet en cause l'architecture actuelle : chacune s'ajouterait comme une extension du module existant plutôt que comme une refonte, ce qui confirme, a posteriori, la solidité des choix de conception retenus dans ce TFE.

## 4.5. Mot de fin

Ce travail de fin d'études m'a permis de mener un projet de bout en bout, de l'entretien avec le commanditaire jusqu'à une application fonctionnelle : recueillir un besoin réel, le traduire en règles de gestion précises, concevoir un modèle de données cohérent, puis l'implémenter avec une pile technique orientée sécurité. Au-delà de l'aspect académique, ce module répond à un besoin concret de Terra Sana ASBL, identifié directement avec Monsieur Seraye durant le stage, et constitue un livrable que l'association pourra effectivement utiliser pour la gestion de ses bénévoles et de ses événements.

Le TFE marque la fin d'un cursus, mais pas celle de ce projet : la présentation du module à Terra Sana et la défense orale restent des étapes à venir, avec l'espoir que ce module dépasse le cadre académique et trouve, à terme, une réelle utilité quotidienne pour les bénévoles et l'équipe de Terra Sana ASBL.

## V. Glossaire

---

Les termes techniques suivants, utilisés à plusieurs reprises dans ce rapport, sont définis ici pour un lecteur non familier du développement web.

| Terme                                | Définition                                                                                                                                                                                                                                  |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| API REST                             | Interface de programmation permettant à deux applications de communiquer via des requêtes HTTP standardisées (GET, POST, PUT, DELETE) échangeant des données au format JSON.                                                                |
| SPA (Single Page Application)        | Application web qui charge une seule page HTML puis met à jour son contenu dynamiquement en JavaScript, sans recharger entièrement la page à chaque navigation. Le frontend React du module en est une.                                     |
| JWT (JSON Web Token)                 | Jeton signé numériquement, transmis à chaque requête pour prouver l'identité de l'utilisateur, sans que le serveur n'ait besoin de conserver une session en mémoire (authentification « stateless »).                                       |
| ORM (Object-Relational Mapping)      | Technique consistant à représenter les tables d'une base de données relationnelle sous forme de classes et d'objets dans le code — Hibernate, utilisé via Spring Data JPA, en est l'implémentation Java la plus répandue.                   |
| DTO (Data Transfer Object)           | Objet dédié au transport de données entre couches (par exemple entre le backend et le frontend), distinct de l'entité stockée en base, permettant de ne jamais exposer directement la structure interne des données.                        |
| Endpoint                             | Une URL précise de l'API REST, associée à une méthode HTTP (ex : POST /api/registrations/event/12), qui déclenche une action ou renvoie une donnée particulière.                                                                            |
| Hachage (mot de passe)               | Transformation irréversible d'un mot de passe en une chaîne de caractères illisible, stockée à la place du mot de passe original — même en cas de fuite de la base de données, le mot de passe réel reste protégé.                          |
| Framework                            | Ensemble structuré d'outils et de conventions (comme Spring Boot ou React) qui prend en charge les aspects techniques répétitifs d'une application, pour que le développeur se concentre sur la logique métier propre à son projet.         |
| Migration (base de données)          | Modification contrôlée et documentée du schéma d'une base de données, permettant de faire évoluer sa structure sans perdre les données déjà existantes.                                                                                     |
| CORS (Cross-Origin Resource Sharing) | Mécanisme de sécurité des navigateurs qui bloque, par défaut, les requêtes entre deux origines différentes (par exemple le frontend sur le port 3000 et le backend sur le port 8080) tant que le serveur ne les autorise pas explicitement. |

## VI. Bibliographie / Webographie

---

### Documentation officielle

Spring. (2026). Spring Boot Reference Documentation (3.5.x).

<https://docs.spring.io/spring-boot/docs/current/reference/html/>

Spring. (2026). Spring Security Reference Documentation. <https://docs.spring.io/spring-security/reference/>

Spring. (2026). Spring Data JPA Reference Documentation.

<https://docs.spring.io/spring-data/jpa/reference/>

Meta Open Source. (2026). React Documentation. <https://react.dev/>

Oracle. (2026). MySQL 8.0 / 9.x Reference Manual. <https://dev.mysql.com/doc/refman/9.1/en/>

Oracle. (2026). Java Platform, Standard Edition 21 Documentation.

<https://docs.oracle.com/en/java/javase/21/>

iText Software. (2026). iText 7 Documentation. <https://itextpdf.com/resources/api-documentation>

MDN Web Docs. (2026). JavaScript Reference. <https://developer.mozilla.org/fr/docs/Web/JavaScript>

### Outils et bibliothèques

jjwt (io.jsonwebtoken). (2026). Java JWT: JSON Web Token for Java. <https://github.com/jwtkt/jjwt>

Chart.js. (2026). Chart.js Documentation. <https://www.chartjs.org/docs/latest/>

Apache Friends. (2026). WampServer — Windows, Apache, MySQL, PHP. <https://www.wampserver.com/>

Apache Software Foundation. (2026). Apache Maven. <https://maven.apache.org/>

GitHub. (2026). GitHub — Where the world builds software. <https://github.com/>

Microsoft. (2026). Visual Studio Code. <https://code.visualstudio.com/>

Postman. (2026). Postman API Platform Documentation. <https://learning.postman.com/>

### Sécurité et bonnes pratiques

OWASP Foundation. (2026). OWASP Top Ten. <https://owasp.org/www-project-top-ten/>

Auth0. (2026). JWT.io — Introduction to JSON Web Tokens. <https://jwt.io/introduction/>

Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT) (RFC 7519). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7519>

Provos, N., & Mazières, D. (1999). A Future-Adaptable Password Scheme (BCrypt). USENIX Annual Technical Conference.

### Ressources pédagogiques

Baeldung. (2026). Baeldung — Java, Spring and Web Development Tutorials. <https://www.baeldung.com/>

E AFC Uccle. (2026). Consignes pour le rapport écrit de l'épreuve intégrée — Bachelier en Informatique de Gestion.

Namur, M.-C. (2025-2026). Notes de cours d'analyse et de modélisation UML — EAFC Uccle.

## VII. Annexes

---

### Annexe A — Script SQL complet

Le script ci-dessous crée l'intégralité des huit tables de la base de données terresana, dans l'ordre imposé par les dépendances de clés étrangères.

```
CREATE DATABASE IF NOT EXISTS terresana CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE terresana;

-- Table : admin (héritée du stage)
CREATE TABLE admin (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(100) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL
);

-- Table : project (héritée du stage)
CREATE TABLE project (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(200) NOT NULL, description TEXT NOT NULL,
  link VARCHAR(500), documentation_link VARCHAR(500),
  image VARCHAR(255), category VARCHAR(255),
  is_active BOOLEAN NOT NULL DEFAULT TRUE, admin_id BIGINT,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_project_admin FOREIGN KEY (admin_id) REFERENCES admin(id)
);

-- Table : blog_post (héritée du stage)
CREATE TABLE blog_post (
  id BIGINT AUTO_INCREMENT PRIMARY KEY, admin_id BIGINT,
  title VARCHAR(255) NOT NULL, content TEXT NOT NULL, image VARCHAR(255),
  is_published BOOLEAN NOT NULL DEFAULT FALSE,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_blogpost_admin FOREIGN KEY (admin_id) REFERENCES admin(id)
);

-- Table : contact_message (héritée du stage)
CREATE TABLE contact_message (
  id BIGINT AUTO_INCREMENT PRIMARY KEY, admin_id BIGINT NULL,
  name VARCHAR(255) NOT NULL, email VARCHAR(255) NOT NULL,
  message VARCHAR(5000) NOT NULL, is_read BOOLEAN NOT NULL DEFAULT FALSE,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_contact_admin FOREIGN KEY (admin_id) REFERENCES admin(id)
);

-- Table : app_users (module TFE)
CREATE TABLE app_users (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  first_name VARCHAR(100) NOT NULL, last_name VARCHAR(100) NOT NULL,
  email VARCHAR(200) NOT NULL UNIQUE, password VARCHAR(255) NOT NULL,
  phone VARCHAR(30), birth_date DATE, gender VARCHAR(20),
  city VARCHAR(255), postal_code VARCHAR(20),
  skills TEXT, availability VARCHAR(255),
  preferred_language VARCHAR(5) NOT NULL DEFAULT 'fr',
  level VARCHAR(10) NOT NULL DEFAULT 'BRONZE',
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  reset_token VARCHAR(255), reset_token_expiry DATETIME,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Table : events (module TFE)
CREATE TABLE events (
  id BIGINT AUTO_INCREMENT PRIMARY KEY, admin_id BIGINT,
  title VARCHAR(255) NOT NULL, description TEXT NOT NULL,
  event_date DATETIME NOT NULL, location VARCHAR(300) NOT NULL,
  max_places INT NOT NULL, status VARCHAR(20) NOT NULL DEFAULT 'OPEN',
  image_url VARCHAR(500),
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_event_admin FOREIGN KEY (admin_id) REFERENCES admin(id)
);
```

```

);

-- Table : registrations (module TFE)
CREATE TABLE registrations (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id BIGINT NOT NULL, event_id BIGINT NOT NULL, validated_by_id BIGINT,
  status VARCHAR(20) NOT NULL DEFAULT 'WAITING', position INT,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT uq_registration_user_event UNIQUE (user_id, event_id),
  CONSTRAINT fk_reg_user FOREIGN KEY (user_id) REFERENCES app_users(id) ON DELETE CASCADE,
  CONSTRAINT fk_reg_event FOREIGN KEY (event_id) REFERENCES events(id) ON DELETE CASCADE,
  CONSTRAINT fk_reg_admin FOREIGN KEY (validated_by_id) REFERENCES admin(id)
);

-- Table : reviews (module TFE)
CREATE TABLE reviews (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id BIGINT NOT NULL, event_id BIGINT NOT NULL,
  rating INT NOT NULL CHECK (rating BETWEEN 1 AND 5), comment TEXT,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT uq_review_user_event UNIQUE (user_id, event_id),
  CONSTRAINT fk_review_user FOREIGN KEY (user_id) REFERENCES app_users(id) ON DELETE CASCADE,
  CONSTRAINT fk_review_event FOREIGN KEY (event_id) REFERENCES events(id) ON DELETE CASCADE
);

-- Index complémentaires : accélèrent les requêtes les plus fréquentes du module
-- (les clés étrangères et contraintes UNIQUE sont déjà indexées automatiquement par MySQL)
CREATE INDEX idx_events_date_location ON events (event_date, location);
-- Supporte checkNoDuplicate (RG-08), appelée à chaque création/modification d'événement
CREATE INDEX idx_events_status ON events (status);
-- Supporte le filtrage par statut sur la liste publique des événements
CREATE INDEX idx_registrations_status ON registrations (event_id, status);
-- Supporte countByEventAndStatus, appelée à chaque affichage des places disponibles

```

Les index sur les clés étrangères et les colonnes UNIQUE sont créés automatiquement par MySQL et n'apparaissent donc pas explicitement ci-dessus. Les trois index complémentaires ajoutés ciblent précisément les requêtes les plus fréquemment exécutées par le module : `idx_events_date_location` accélère la vérification de doublon (RG-08), appelée à chaque création ou modification d'événement, et `idx_registrations_status` accélère le comptage des places disponibles, recalculé à chaque affichage d'un événement plutôt que stocké (section 2.2.3).

## **Annexe B — Note sur les captures d'écran**

Les captures d'écran présentées en section 3.4 ont été réalisées en environnement local (localhost), à partir de données de démonstration créées pour les besoins de ce rapport — notamment un compte bénévole de test (« Camille Dupont ») créé via le formulaire d'inscription public pour illustrer l'espace personnel bénévole en conditions réelles plutôt qu'à l'état vide.

## **Annexe C — Dépôt GitHub**

Le code source complet du site Terra Sana, incluant le module de gestion des bénévoles et des événements développé dans le cadre de ce TFE, est disponible à l'adresse suivante :

<https://github.com/sidanne/terra-sana-website>

# **Alain Youndjeu Tchouapi**

*Module de gestion des bénévoles et des événements — Terra Sana ASBL*

**TERRA SANA ASBL**  
**EAFC UCCLE**